

This is a copy of a shared ChatGPT chat. It won't be added to memory.

Report conversation

Yesterday 7:32 PM

Nina ujuzi wa Data analysis, full stack developer na nimesoma Bachelor of Official Statistics. Nahitaji kutengeneza system na App ambayo itawasahidia watu kufanya analysis za data zao Kwa urahisi zaidi, ikiwa na ku clean data, manipulate, analysis na kutoa majibu ya kitakwimu kulingana na data alizooewa. Pia iwe nauwezo wa kuchora graph vizuri, iwe fast hata Kwa big data analysis.

Kisha tujadiliane hatua nyingine

Ndiyo. Hii ni **idea nzuri sana**, hasa kwa background yako ya **Official Statistics + Data Analysis + Full-Stack Development**. Unachokielezea si dashboard ya kawaida tu; tunaweza kuijenga kama **Data Analysis Platform** ambayo mtumiaji ana-upload data, system inaelewa, ina-clean, ina-analyze, ina-visualize na kusaidia kutafsiri matokeo.

Ningependekeza tusianze na coding moja kwa moja. Kwanza tujenge **architecture na product concept** vizuri, kwa sababu performance ya *big data* na statistical analysis itategemea maamuzi tunayofanya mwanzo.

1. Mfumo tunaolenga kujenga

Kwa mfano mtumiaji ana-upload:

- `.csv`
- `.xlsx`
- `.xls`
- `.json`
- baadaye `.sav` (SPSS)
- `.dta` (Stata)
- `.parquet`

Kisha mfumo umpeleke kwenye workflow kama:

Upload → Inspect → Clean → Transform → Analyze → Visualize → Interpret → Export

Mfano:

User ana-upload survey data yenye respondents 500,000.

System inaweza kumwambia:

- Dataset ina **500,000 observations**
- Variables: **42**
- Missing values: **8.4%**
- Duplicate records: **1,245**
- Outliers zilizogunduliwa: ...
- Categorical variables: ...
- Numerical variables: ...
- Date variables: ...
- Potential ID variables: ...

Halafu user anaweza kusema:

"Nataka kujua relationship kati ya education level na household income."

System inaweza kupendekeza:

- Cross-tabulation
- Chi-square test kama variables zinafaa
- Mean/median comparison
- Correlation ikiwa appropriate
- Regression ikiwa inahitajika

Na matokeo yawe na **table + graph + statistical interpretation**.

2. Modules kuu za system

Ningegawanya platform katika modules hizi:

A. Data Import

Mtumiaji anaweza:

- Upload file
- Connect database
- Import from API

- Google Sheets
- SQL database
- Cloud storage

B. Data Profiling

Hii ni muhimu sana.

System automatically ichunguze:

```
Rows
Columns
Data types
Missing values
Duplicates
Unique values
Outliers
Invalid values
Distributions
Variable relationships
```

C. Data Cleaning

User aweze kufanya:

- Remove duplicates
- Handle missing values
- Rename variables
- Change data types
- Remove columns
- Filter rows
- Detect outliers
- Replace values
- Standardize text
- Date formatting
- Recode variables

Mfano:

```
Gender

M
Male
male
MALE
F
```

Female

System inaweza kupendekeza:

Male

Female

3. Data Manipulation

User aweze kufanya operations kama:

Filter
Sort
Group By
Merge
Join
Aggregate
Pivot
Unpivot
Calculate Column
Recode
Normalize
Standardize
Sampling

Mfano:

Income > 500,000
AND
Age >= 18

bila user kuandika code.

Lakini pia tunaweza kuwa na:

Advanced Mode → Python / R / SQL

kwa users wenye advanced knowledge.

4. Statistical Analysis Engine

Hapa ndipo background yako ya Statistics itakuwa advantage kubwa.

Tunaweza kuwa na categories:

Descriptive Statistics

- Mean
- Median
- Mode
- Variance
- Standard deviation
- Range
- Quartiles
- Percentiles
- CV
- Skewness
- Kurtosis

Inferential Statistics

- t-test
- paired t-test
- ANOVA
- Chi-square
- correlation
- confidence intervals
- proportion tests
- non-parametric tests

Regression

- Linear regression
- Multiple regression
- Logistic regression
- Poisson regression
- Negative binomial
- Panel models
- Time-series models

Advanced Statistics

Baadaye:

- PCA
- Factor analysis
- Cluster analysis
- Survival analysis
- Forecasting
- Time series

- Multilevel models
- Structural equation models

Na muhimu zaidi, system isiangelie tu:

"Which test can I run?"

bali iweze kusaidia:

"Which statistical method is appropriate for my research question and variables?"

5. AI Statistical Assistant

Hii inaweza kuwa sehemu kubwa sana ya product.

Mfano user anaandika:

"Nataka kujua kama elimu ina relationship na income."

AI ichunguze dataset na kusema:

```
Research question detected:  
Relationship between education level and income.
```

```
Variables identified:  
Education → categorical  
Income → continuous
```

```
Recommended analyses:  
1. Descriptive statistics  
2. Boxplot by education level  
3. ANOVA  
4. Post-hoc comparison
```

Kisha user anabonyeza:

Run Analysis

6. Graph & Visualization Engine

Tutahitaji visualization nzuri sana:

- Bar chart
- Histogram

- Pie/donut
- Line chart
- Scatter plot
- Boxplot
- Violin plot
- Heatmap
- Area chart
- Stacked bar
- Population pyramid
- Maps
- Time-series charts

Na muhimu:

Interactive graphs

Mtumiaji aweze:

- zoom
- filter
- hover
- download
- change chart type
- change labels
- customize colors
- export PNG/SVG/PDF

7. Automated Interpretation

Hii itakuwa tofauti na tools nyingi.

Badala ya kutoa:

```
p-value = 0.003
```

system inaweza kutoa:

The analysis indicates a statistically significant association between education level and household income ($\chi^2 = \dots$, $p < 0.05$).

Lakini lazima iwe **controlled statistical interpretation**, si AI kubuni conclusion.

Tunaweza kuweka **statistical validation layer** kabla AI haijaandika interpretation.

8. Reports

User aweze kutengeneza:

Statistical Report

```
Title
Introduction
Dataset description
Data cleaning
Descriptive statistics
Analysis
Tables
Figures
Interpretation
Conclusion
```

Export:

- PDF
- Word
- Excel
- PowerPoint
- HTML

Hii inaweza kuwa muhimu sana kwa:

- Students
- Researchers
- NGOs
- Government
- Businesses
- Consultants
- Universities
- Data analysts

9. Big Data Architecture

Hapa ndipo tunapaswa kuwa makini.

Sitashauri system iwe:

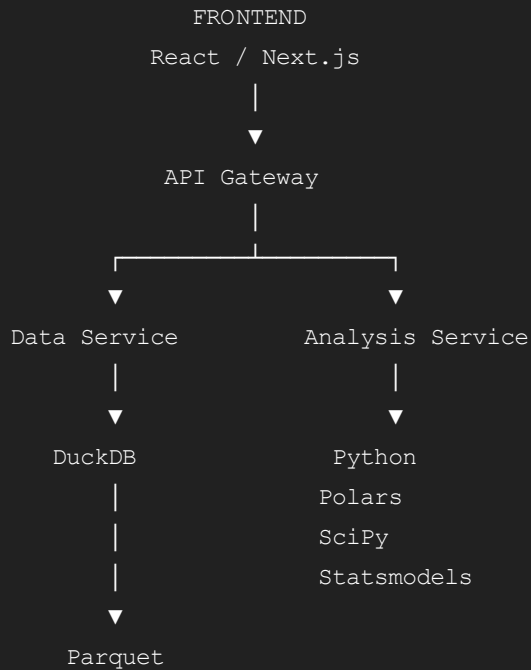
```
React
↓
Node.js
```

↓
Pandas

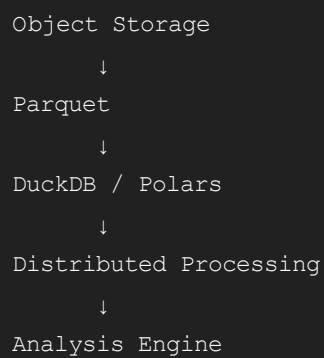
kwa kila analysis.

Kwa data kubwa inaweza kuwa bottleneck.

Badala yake tunaweza kutumia architecture kama:



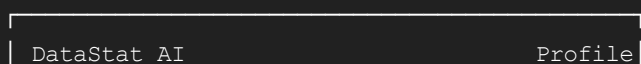
Na kwa very large datasets baadaye:



Hii itatusaidia kufanya mfumo uwe **fast** bila kulazimisha data yote iingie RAM.

10. Frontend

Kwa experience ya user ningependekeza interface yenye maeneo kama:



Dashboard	Workspace
Datasets	
	Data Preview
Cleaning	
Transform	
Analysis	
Visualize	
Reports	

Kuna concept nzuri hapa:

One Dataset → One Workspace

User aki-open dataset anaingia kwenye workspace moja ambayo ina:

Data → Cleaning → Analysis → Visualization → Report

Hii inafanya workflow iwe rahisi kuelewa.

11. Technology stack ninayopendekeza

Kwa skills zako za Full Stack:

Frontend

React / Next.js

-

TypeScript

-

Tailwind CSS

-

Plotly / Apache ECharts

Backend/API

Python + FastAPI

Kwa sababu statistical ecosystem ya Python ni kubwa.

```
FastAPI
├── Pandas
├── Polars
├── NumPy
├── SciPy
├── Statsmodels
└── Scikit-learn
```

Kwa heavy processing:

Polars + DuckDB + Parquet

Database

Kwa application data:

PostgreSQL

Kwa datasets:

Object Storage + Parquet

Kwa local/analytical querying:

DuckDB

12. AI layer

AI isiwe ndiyo inayofanya statistical calculations.

Badala yake:

```
User Question
    ↓
AI understands question
    ↓
Statistical Planning Engine
    ↓
Validation
    ↓
Analysis Engine
    ↓
Actual numerical result
```

↓
AI explains result

Hii ni muhimu sana.

Kwa mfano AI isiamue yenyewe kwamba:

p-value = 0.021

Calculation ifanywe na statistical engine.

AI ichukue **actual result** na kuielezea.

13. Security

Kwa sababu watu wata-upload datasets zao, security itakuwa muhimu sana.

Tunahitaji:

- Authentication
- Role-based access
- Dataset isolation
- Encryption
- Secure file upload
- Audit logs
- Automatic deletion options
- API security
- Rate limiting

Na hasa:

User A asiwahi kuona dataset ya User B.

14. Business model

Baadaye tunaweza kuwa na:

Free

Limited datasets
Limited rows
Basic statistics
Basic charts

Professional

- Large datasets
- Advanced statistics
- AI assistant
- Reports
- Export

Organization

- Team workspace
- Shared datasets
- User management
- Advanced security
- API

Enterprise

- Private deployment
- Custom integrations
- Dedicated infrastructure

15. Lakini tusijenge kila kitu mara moja

Hili ndilo jambo muhimu zaidi.

Mimi ningependekeza tuanze na **MVP**.

MVP v1

Iwe na:

1. User authentication
2. Upload CSV/XLSX
3. Dataset preview
4. Automatic data profiling
5. Basic cleaning
6. Data transformation
7. Descriptive statistics
8. Basic statistical tests
9. Interactive charts
10. AI statistical assistant
11. Export results

Baada ya hiyo kufanya kazi vizuri:

v2

SPSS
Stata
SQL
Large datasets
Advanced regression
Advanced visualization
Reports

v3

Team collaboration
Cloud databases
API
Mobile app
Enterprise
Advanced ML

Jambo la kwanza ningependekeza tulifanye

Kabla hatujaandika hata line moja ya code, **tuchore architecture kamili ya system.**

Tunaweza kuanza hatua inayofuata kwa kubuni:

① **Product name/concept** → ② **User workflow** → ③ **Features zote** → ④ **System architecture** → ⑤ **Database schema** → ⑥ **Statistical engine** → ⑦ **AI architecture** → ⑧ **Big-data architecture** → ⑨ **UI pages** → ⑩ **Technology stack** → ⑪ **MVP development roadmap.**

Kwa kuwa una background ya **Official Statistics**, tunaweza pia kufanya platform hii iwe na kitu cha kipekee: **statistical decision engine** inayosaidia mtumiaji kuchagua analysis sahihi kulingana na aina ya variables, research question, assumptions na study design—badala ya kuwa Excel/SPSS clone tu.

Sawa. Endelea

Sawa. Tuiendeleo kwa ku-design **mfumo wenyewe** kama tunajenga product halisi, si project ya kawaida ya CRUD.

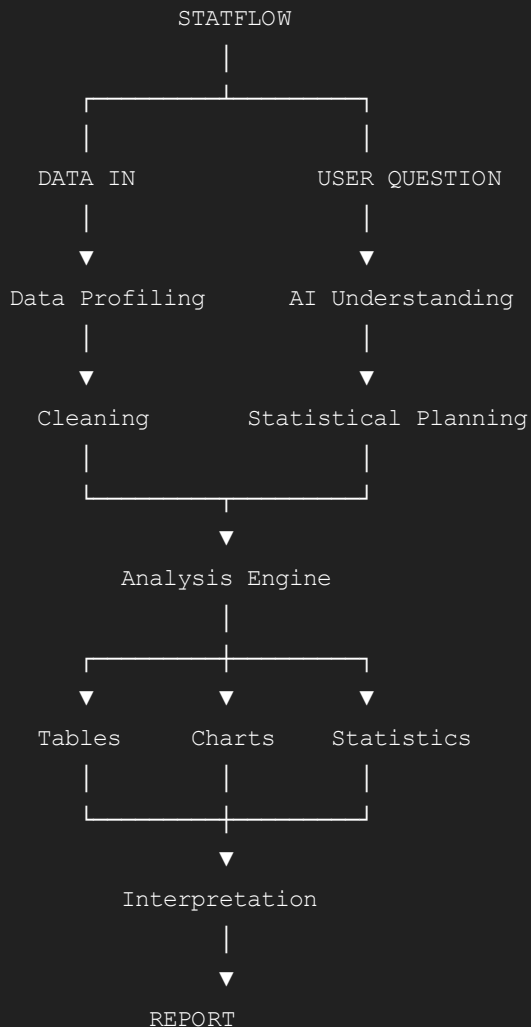
1. Concept ya Platform

Kwa sasa tuipe working name:

StatFlow

From Raw Data to Statistical Insight

Core idea:



Hii inafanya system iwe na **workflow moja inayoeleweka** kutoka data mbichi mpaka insight.

2. User Workflow

Mtumiaji akiingia, experience iwe:

Step 1 — Create Project

New Analysis

Mfano:

Household Survey 2026

Step 2 — Upload Data

Upload Dataset

Supported:

CSV
XLSX
JSON

Baadaye:

SAV
DTA
Parquet
SQL
API

Step 3 — Automatic Profiling

Mara tu file inapopakiwa, system isianze analysis bila kuelewa data.

lonyeshe:

Dataset Overview

Rows	250,000
Columns	38
Missing values	4.8%
Duplicates	1,203
Numeric variables	19
Categorical variables	15
Date variables	4

Kisha kila variable:

Variable	Type	Missing	Unique	Suggested Role
age	Numeric	0.2%	76	Predictor

Variable	Type	Missing	Unique	Suggested Role
gender	Categorical	0%	2	Group
income	Numeric	3.1%	18,230	Outcome
education	Categorical	0.4%	6	Predictor

Hii itakuwa **Data Intelligence Layer**.

3. Data Cleaning Studio

Badala ya user kutumia code, apewe visual workflow.

Mfano:

Column: age

Detected issues:

- ⚠ 23 missing values
- ⚠ 4 impossible values
- ⚠ 2 extreme values

Recommended actions:

- [Remove rows]
- [Impute missing]
- [Replace invalid values]
- [Investigate outliers]

User akifanya action, system itengeneze **operation history**.

Mfano:

1. Removed duplicates
2. Replaced "M" → "Male"
3. Imputed missing age
4. Removed invalid income values

Muhimu: **original dataset isibadilishwe**.

Tunafanya:

Original Data

↓

Transformation Layer

↓
Clean Dataset

Hii itaruhusu user kurudi nyuma (**undo/versioning**).

4. Analysis Workspace

Hapa ndipo platform itakuwa powerful.

Workspace iwe na tabs:

Overview
Data
Clean
Transform
Analyze
Visualize
AI Assistant
Report

User anaweza kwenda moja kwa moja sehemu anayohitaji.

5. Statistical Decision Engine

Hii ndiyo sehemu ambayo ningependa iwe **signature feature** ya StatFlow.

User anaweza kuandika:

"Nataka kujua kama income inatofautiana kulingana na education level."

Engine ichambue:

```
Question
↓
Identify variables
↓
Determine variable types
↓
Identify objective
↓
Check assumptions
↓
Select candidate methods
↓
Run appropriate method
```

Kwa mfano:

```
Outcome:
Income → Continuous

Grouping variable:
Education → Categorical

Groups:
6
```

System inaweza kuonyesha methods zinazofaa kulingana na data na assumptions.

6. Statistical Analysis Categories

Descriptive

```
Frequency
Percentage
Mean
Median
Mode
Variance
SD
CV
Percentiles
Skewness
Kurtosis
```

Association

Pearson correlation
Spearman correlation
Chi-square
Cramér's V

Comparison

One-sample t-test
Independent t-test
Paired t-test
ANOVA
Mann-Whitney
Wilcoxon
Kruskal-Wallis

Regression

Linear regression
Multiple linear regression
Logistic regression
Poisson regression
Negative binomial

Time Series

Trend
Moving average
Decomposition
Forecasting

Baadaye tunaweza kuongeza:

PCA
Factor Analysis
Cluster Analysis
Survival Analysis
Panel Data
Multilevel Models

7. Assumption Checking

Hapa tutatofautisha platform yetu na calculator ya kawaida.

Mfano user anataka Linear Regression.

System isifanye tu:

Run Regression

lanze:

Model Diagnostics

- ✓ Missing values checked
- ✓ Variable types checked
- ✓ Multicollinearity checked
- ✓ Residual diagnostics
- ✓ Homoscedasticity
- ✓ Normality diagnostics
- ✓ Influential observations

Kisha:

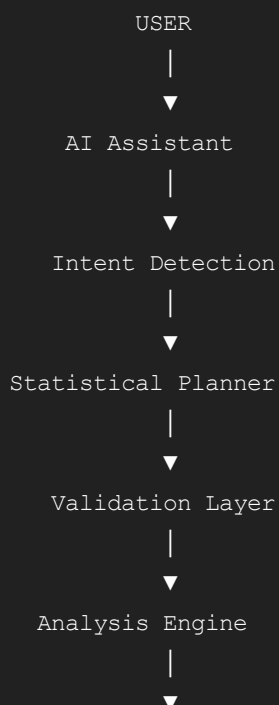
Model diagnostics indicate potential heteroscedasticity. Consider robust standard errors or an alternative specification.

Hii ni sehemu ambayo background yako ya statistics inaweza kuwa advantage kubwa.

8. AI Assistant Architecture

AI haitafanya calculation yenyewe.

Tutakuwa na layers:



Verified Results



AI Explanation

Kwa hiyo:

AI = kuelewa + kusaidia + kuelezea

lakini:

Statistical Engine = kufanya calculation.

Hii ni muhimu ili kupunguza hallucinated statistical results.

9. Visualization Engine

User akimaliza analysis anaweza kubonyeza:

Visualize

System inaweza kusema:

"Based on your selected variables, these chart types are suitable."

Mfano:

Income vs Education

Recommended:

[Box Plot]
[Violin Plot]
[Mean + CI]

User anaweza pia kuchagua manually:

Bar
Line
Pie
Histogram
Scatter
Boxplot
Violin
Heatmap
Area
Stacked Bar

10. Automatic Chart Intelligence

Mfano user ana variables:

Month
Sales

System itambue:

Month → Time
Sales → Numeric

na kupendekeza:

Line chart

Kwa:

Gender
Education

itambue:

Categorical × Categorical

na kupendekeza:

Grouped/Stacked Bar

Kwa:

Age
Income

itambue:

Numeric × Numeric

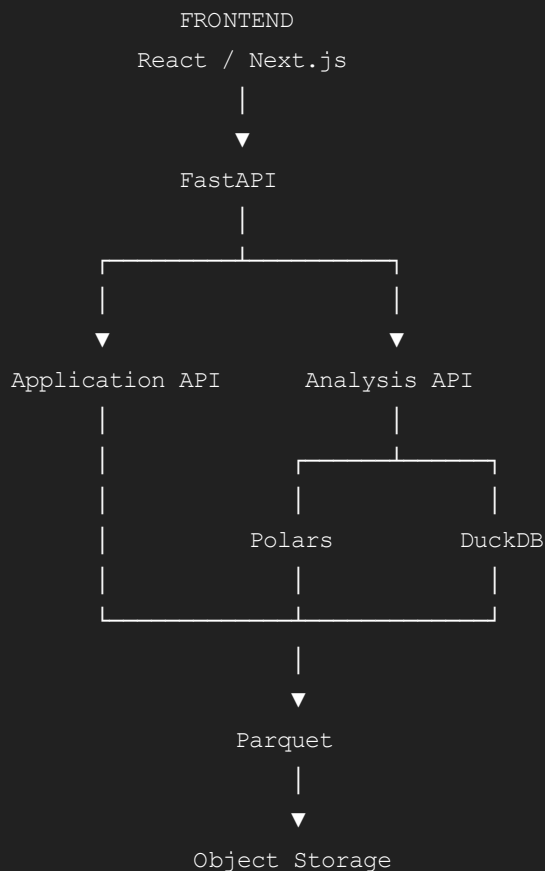
na kupendekeza:

Scatter Plot

11. Big Data Architecture

Hii ni muhimu sana kwa requirement yako ya kuwa fast.

Ningepanga hivi:



Why Polars?

Kwa data processing tunataka kutumia **columnar processing** na operations zinazofaa kwa datasets kubwa.

Why DuckDB?

Kwa analytical SQL queries kwenye files kama Parquet, DuckDB inaweza kufanya querying bila kuhitaji database kubwa ya transactional.

Why Parquet?

Kwa datasets kubwa:

CSV → storage

Parquet → analytical processing

Hivyo user anaweza ku-upload CSV lakini backend inaweza ku-convert kwenye optimized format.

12. Database Architecture

Tutenganishe:

PostgreSQL

Kwa system information:

```
users
organizations
projects
datasets
dataset_versions
analyses
analysis_results
charts
reports
subscriptions
audit_logs
```

Object Storage

Kwa actual datasets:

```
datasets/
  user-id/
    project-id/
      original/
      processed/
      exports/
```

Analytical Storage

```
Parquet
```

Hii separation ni muhimu.

Usiweke millions of data rows kwenye PostgreSQL kwa kila user dataset ikiwa tunataka scalable analytical workloads.

13. Backend Services

Badala ya backend moja kubwa, architecture iwe modular:

```
backend/
|
```

```
├─ auth/
├─ users/
├─ projects/
├─ datasets/
├─ profiling/
├─ cleaning/
├─ transformation/
├─ statistics/
├─ visualization/
├─ reports/
├─ ai/
├─ exports/
└─ jobs/
```

Baadaye tukikua tunaweza kuzi-separate kuwa services.

Kwa MVP bado tunaweza kuwa na **modular monolith**, ambayo ni rahisi sana ku-develop kuliko kuanza na microservices nyingi.

14. Frontend Structure

Kwa React/Next.js:

```
src/
|
├─ app/
├─ components/
|   ├─ Layout/
|   ├─ Sidebar/
|   ├─ Topbar/
|   ├─ DataTable/
|   ├─ Chart/
|   ├─ DatasetCard/
|   ├─ StatisticCard/
|   └─ AIChat/
|
├─ features/
|   ├─ datasets/
|   ├─ cleaning/
|   ├─ transformation/
|   ├─ analysis/
|   ├─ visualization/
|   └─ reports/
|
├─ services/
├─ hooks/
├─ utils/
└─ types/
```

Kwa style uliyokuwa unapendelea kwenye projects zako nyingine, tunaweza pia kufanya kila reusable component iwe folder yake yenye:

```
DataTable/  
  DataTable.jsx  
  DataTable.css
```

lakini kwa project hii ningependekeza **TypeScript + component-level styling strategy** ili scalability iwe bora.

15. MVP ya kwanza kabisa

Tusijenge kila statistical method.

MVP-1

Data

- CSV
- XLSX

Profiling

- rows/columns
- data types
- missing values
- duplicates
- unique values
- distributions

Cleaning

- missing values
- duplicates
- rename
- filter
- recode
- data type conversion

Statistics

- frequency
- percentage
- mean
- median
- SD

- variance
- min/max
- correlation
- t-test
- chi-square
- ANOVA
- linear regression

Visualization

- bar
- line
- histogram
- scatter
- boxplot
- pie
- heatmap

AI

- ask about dataset
- suggest analysis
- explain results

Export

- CSV
- Excel
- PNG
- PDF

16. Hatua inayofuata

Sasa tumeshapata **product architecture**. Hatua inayofuata ninglefanya ni muhimu zaidi:

Tutengeneze Technical Blueprint v1

Itakuwa na:

1. Complete system architecture
2. Frontend architecture
3. Backend architecture
4. PostgreSQL database schema
5. Dataset storage architecture
6. Statistical engine architecture
7. AI architecture

8. API endpoints
9. Authentication & authorization
10. Big-data processing pipeline
11. Job/queue system
12. Folder structure
13. Development phases
14. MVP → V2 → V3 roadmap

Baada ya blueprint hiyo, tunaweza kuanza **database schema + API design**, halafu ndipo tuanze coding ya MVP.

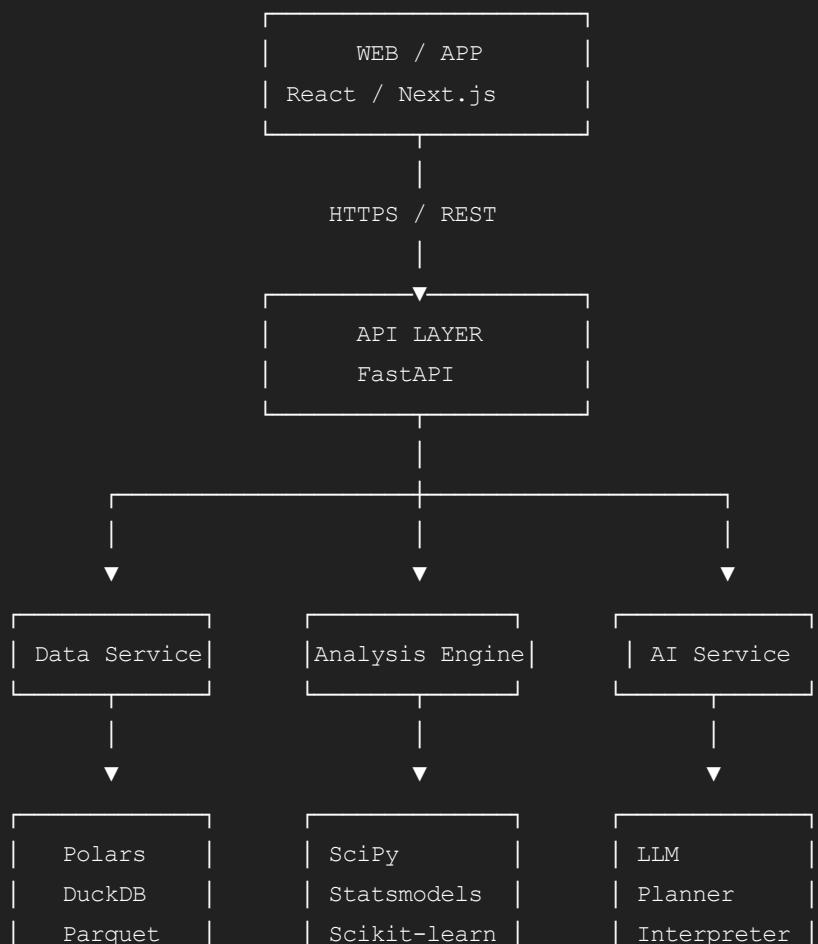
Endelea

TECHNICAL BLUEPRINT — STATFLOW v1

Tutengeneza mfumo kwa principle moja kuu:

Easy for beginners, powerful for statisticians, scalable for big data.

1. Overall Architecture





2. Kwa nini tusitumie architecture ya kawaida?

Mfano ambao ningependa tuue mapema ni:

```
React
  ↓
Node.js
  ↓
PostgreSQL
  ↓
Pandas
```

Kwa dataset ndogo itafanya kazi.

Lakini tukija kwenye:

```
2 million rows
10 million rows
50 million rows
```

tutaanza kupata:

- memory problems
- slow processing
- timeout
- expensive server resources

Ndiyo maana tunatenganisha:

Transactional data

→ PostgreSQL

Analytical data

→ Parquet + DuckDB + Polars

AI

→ Separate AI layer

3. Database Schema

PostgreSQL haitahifadhi kila row ya dataset.

Itahifadhi **metadata kuhusu dataset**.

Users

```
users
-----
id
name
email
password_hash
role
created_at
updated_at
```

Organizations

Kwa future ya teams:

```
organizations
-----
id
name
owner_id
plan
created_at
```

Projects

```
projects
-----
id
organization_id
```

```
name
description
created_by
created_at
updated_at
```

Mfano:

```
Project:
Katavi Smallholder Farmers Survey
```

4. Datasets

```
datasets
-----
id
project_id
name
original_filename
file_type
storage_path
row_count
column_count
file_size
status
created_by
created_at
```

Status inaweza kuwa:

```
UPLOADING
PROCESSING
READY
FAILED
ARCHIVED
```

5. Dataset Versions

Hii ni muhimu sana.

```
dataset_versions
-----
id
dataset_id
```

```
version_number
storage_path
row_count
created_by
operation_id
created_at
```

Kwa mfano:

```
Version 1
Original dataset

Version 2
Removed duplicates

Version 3
Handled missing values

Version 4
Created income_group
```

Kwa hiyo user anaweza kurudi kwenye version yoyote.

6. Variables

Metadata ya columns:

```
variables
-----
id
dataset_version_id
name
label
data_type
semantic_type
is_numeric
is_categorical
is_date
missing_count
unique_count
min_value
max_value
created_at
```

Mfano:

```
age
data_type = integer
```

```
semantic_type = continuous
```

au:

```
gender
data_type = string
semantic_type = nominal
```

7. Analysis Table

```
analyses
-----
id
project_id
dataset_version_id
analysis_type
name
configuration
status
created_by
created_at
completed_at
```

Mfano:

JSON

```
{
  "analysis_type": "linear_regression",
  "dependent": "income",
  "independent": [
    "age",
    "education",
    "farm_size"
  ]
}
```

8. Analysis Results

```
analysis_results
-----
id
analysis_id
result_type
```

```
result_data
created at
```

`result_data` inaweza kuwa JSON.

Mfano:

JSON

```
{
  "r_squared": 0.61,
  "adjusted_r_squared": 0.59,
  "coefficients": [],
  "p_values": [],
  "confidence_intervals": []
}
```

9. Charts

```
charts
-----
id
analysis_id
chart_type
configuration
image_path
created_at
```

Configuration:

JSON

```
{
  "x": "education",
  "y": "income",
  "group": "gender",
  "title": "Income by Education Level"
}
```

10. Reports

```
reports
-----
id
```

```
project_id
title
format
content
status
created_by
created_at
```

Formats:

```
PDF
DOCX
XLSX
PPTX
HTML
```

11. API Architecture

FastAPI endpoints zinaweza kuwa:

```
/api/v1
```

Authentication

```
POST /auth/register
POST /auth/login
POST /auth/refresh
POST /auth/logout
```

Projects

```
GET    /projects
POST   /projects
GET    /projects/{id}
PATCH /projects/{id}
DELETE /projects/{id}
```

Datasets

```
POST /datasets/upload
GET  /datasets
```

```
GET    /datasets/{id}
DELETE /datasets/{id}
```

Profiling

```
POST /datasets/{id}/profile
GET  /datasets/{id}/profile
```

Cleaning

```
POST /datasets/{id}/clean
GET  /datasets/{id}/operations
POST /datasets/{id}/undo
```

Transformation

```
POST /datasets/{id}/transform
POST /datasets/{id}/filter
POST /datasets/{id}/aggregate
POST /datasets/{id}/join
```

Analysis

```
POST /analysis/descriptive
POST /analysis/correlation
POST /analysis/ttest
POST /analysis/chi-square
POST /analysis/anova
POST /analysis/regression
```

Visualization

```
POST /charts
GET  /charts/{id}
POST /charts/export
```

AI

```
POST /ai/ask
POST /ai/suggest-analysis
POST /ai/explain-result
```

12. Analysis Engine

Hapa tunatengeneza abstraction ambayo itaruhusu kuongeza statistical methods bila kuvuruga backend.

```
analysis_engine/
|
├─ base/
|   ├── analyzer.py
|   ├── validator.py
|   └─ result.py
|
├─ descriptive/
|   ├── frequency.py
|   ├── summary.py
|   └─ distribution.py
|
├─ comparison/
|   ├── ttest.py
|   ├── anova.py
|   └─ nonparametric.py
|
├─ association/
|   ├── correlation.py
|   └─ chi_square.py
|
├─ regression/
|   ├── linear.py
|   ├── logistic.py
|   └─ poisson.py
|
└─ timeseries/
    ├── decomposition.py
    └─ forecasting.py
```

13. Standard Analysis Interface

Kila analysis ifuate interface inayofanana.

Mfano:

Python

Run

```
class StatisticalAnalyzer:

    def validate(self, data, config):
        pass

    def run(self, data, config):
        pass

    def interpret(self, result):
        pass
```

Kwa hiyo tukiongeza:

ANOVA

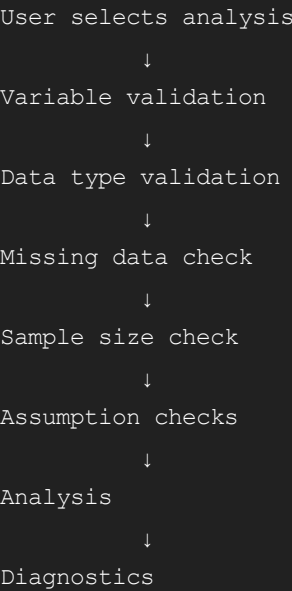
au:

Logistic Regression

hazitabadilisha architecture nzima.

14. Statistical Validation Layer

Hii itakuwa moja ya modules muhimu zaidi.



↓

Mfano:

User anajaribu Pearson correlation.

System iangalie:

- ✓ Both variables numeric
- ✓ Sufficient observations
- ⚠ Extreme outliers detected
- ⚠ Non-linear relationship suspected

Badala ya kumpa user result bila context.

15. AI Statistical Planner

Hii ndiyo engine ya AI.

Mfano:

User:

"Nataka kujua kama gender ina influence kwenye income."

AI itatoa structured plan:

JSON

```
{
  "question_type": "group_comparison",
  "outcome": "income",
  "predictor": "gender",
  "outcome_type": "continuous",
  "predictor_type": "categorical",
  "candidate_methods": [
    "independent_t_test"
  ]
}
```

Kisha **statistical validator** ndiyo inaamua kama method hiyo inaruhusiwa.

AI haiwezi ku-send arbitrary Python code moja kwa moja kwenye production environment.

16. AI Safety Architecture

Tunaweka:

```
AI
↓
Structured JSON
↓
Schema Validator
↓
Statistical Validator
↓
Execution Engine
```

Sio:

```
AI
↓
Python code
↓
Execute
```

Hii inapunguza security risk na hallucination.

17. Job Queue

Analysis kubwa hazifanywi ndani ya HTTP request.

Mfano:

```
User clicks Analyze
↓
API creates job
↓
Queue
↓
Worker
↓
Analysis
↓
Store result
↓
Notify frontend
```

Technology inaweza kuwa:

Redis + Celery

au alternative ya kisasa kulingana na deployment environment.

18. Big Data Processing

Kwa data ndogo:

< 100 MB

tunaweza kufanya interactive processing.

Data kubwa:

100 MB - several GB

tutumie:

Polars
DuckDB
Parquet
Background Jobs

Na tukifikia data kubwa sana:

Distributed Processing

tunaweza kuongeza engine kama Spark/alternative bila kubadilisha frontend.

19. Frontend UX

Dashboard:

StatFlow	 Account	
Dashboard	Welcome back	
Projects	Projects Datasets	
Datasets		
Analysis	Recent Projects	
Visualize		

Reports	
AI Assistant	
Settings	

20. Dataset Workspace

Hii ndiyo screen muhimu zaidi.

Dataset: Household Survey 2026			
Overview	Data	Clean	Transform Analyze
250,000 rows		38 variables	
Variable	Type	Missing	Unique
age	Numeric	0.2%	76
gender	Category	0%	2
income	Numeric	3.1%	18230

21. "Ask Data" Interface

Hii naona inaweza kuwa feature maarufu sana.

User aandike:

Show me the average income by region.

System:

Understanding request...

- ✓ Dataset identified
- ✓ Region → categorical
- ✓ Income → numeric

Analysis:
Group-by mean

[Run Analysis]

Result:

Region	Mean Income
Katavi	542,000
Dodoma	621,000
...	

na chart automatic.

22. Advanced Mode

Kwa statistician/developer:

Simple Mode

|
└─ Visual controls

Advanced Mode

|
├─ SQL
├─ Python
└─ Statistical syntax

Kwa mfano:

SQL

```
SELECT region, AVG(income)
FROM dataset
GROUP BY region;
```

Hii itafanya platform isiwe beginner-only.

23. Project Folder Structure

Ningependekeza repository iwe:

```
statflow/
|
├─ frontend/
```

```
|   └─ src/
|       └─ app/
|           └─ components/
|               └─ features/
|                   └─ services/
|                       └─ hooks/
|                           └─ utils/
|                               └─ types/
|                                   └─ package.json
|
|   └─ backend/
|       └─ app/
|           └─ api/
|               └─ core/
|                   └─ models/
|                       └─ schemas/
|                           └─ services/
|                               └─ repositories/
|                                   └─ workers/
|                                       └─ main.py
|                                           └─ requirements.txt
|
|   └─ analysis-engine/
|       └─ descriptive/
|           └─ inferential/
|               └─ regression/
|                   └─ timeseries/
|                       └─ validation/
|                           └─ common/
|
|   └─ infrastructure/
|       └─ docker/
|           └─ nginx/
|               └─ scripts/
|
|   └─ docs/
|
|   └─ tests/
|
|   └─ docker-compose.yml
|   └─ README.md
```

24. Development Roadmap

Phase 1 — Foundation

Authentication

Users

Projects
Dataset upload
PostgreSQL
Object storage

Phase 2 — Data Engine

File parsing
Profiling
Data types
Missing values
Duplicates
Data preview

Phase 3 — Cleaning

Filter
Sort
Rename
Recode
Missing handling
Transformation
Versioning

Phase 4 — Statistics

Descriptive
Correlation
Chi-square
t-tests
ANOVA
Regression

Phase 5 — Visualization

Charts
Interactive charts
Chart recommendations
Export

Phase 6 — AI

Ask Data
Analysis recommendation

Phase 7 — Reports

PDF
Word
Excel
PowerPoint

Phase 8 — Scale

Parquet
DuckDB
Polars
Queue
Workers
Caching
Large datasets

25. Feature ambayo ningependa tuiongeze mapema

Kwa sababu umefundishwa **Official Statistics**, tunaweza kutengeneza module ambayo platforms nyingi hazijengi vizuri:

Survey Analysis

Badala ya system kuwa general data analysis pekee, iwe na support ya:

Survey weights
Sampling weights
Strata
Clusters / PSU
Finite population correction
Design effects
Weighted estimates
Confidence intervals

Hii inaweza kuwa muhimu sana kwa:

- household surveys
- government surveys
- NGO surveys
- academic research
- population studies

Na inaweza kuwa moja ya differentiators kubwa ya product.

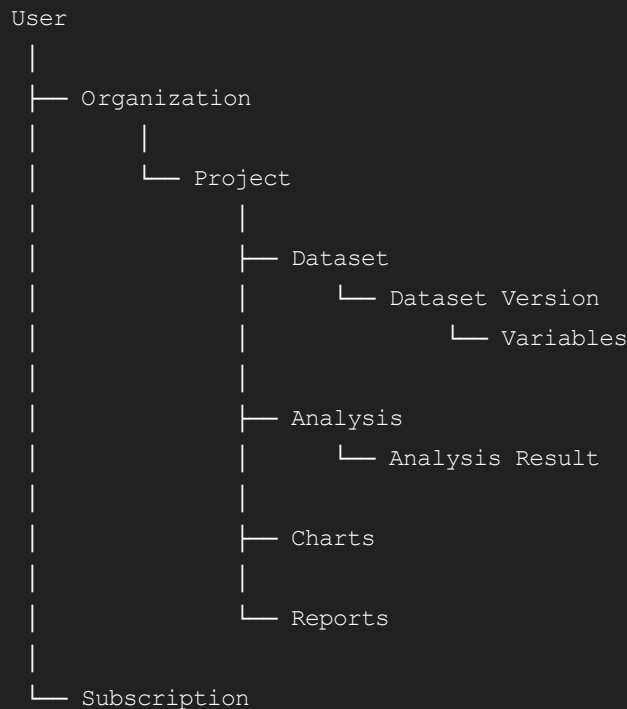
26. Hatua yetu inayofuata kabisa

Sasa architecture iko kwenye kiwango cha kutosha kuanza **Database + API contract**.

Ningependekeza hatua inayofuata iwe:

STATFLOW DATABASE DESIGN v1

Tutachora ERD kamili:



Kisha tutaandika **PostgreSQL schema halisi**, relationships, indexes, constraints na API request/response structures.

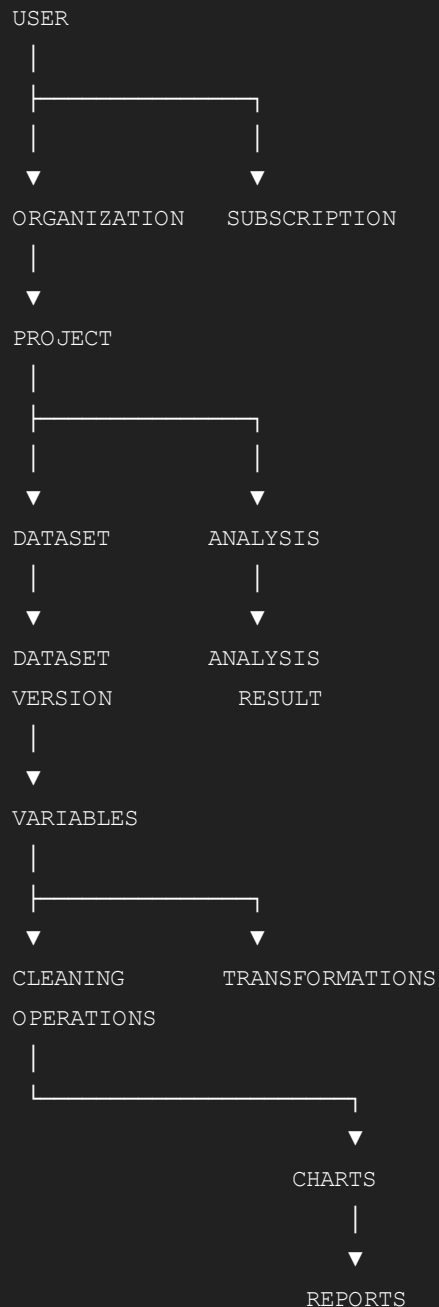
Baada ya hapo ndipo tutakuwa tayari kuanza kujenga **MVP codebase ya frontend + backend + analysis engine** bila kubadilisha msingi wa system katikati ya development.

Endelea

Sawa. Sasa tuingie kwenye **Database Design + API Contract v1**. Hapa tunaanza kugeuza idea yetu kuwa mfumo unaoweza ku-code moja kwa moja.

27. STATFLOW — Database ERD v1

Muundo mkuu:



28. users

SQL

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  full_name VARCHAR(150) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  role VARCHAR(30) DEFAULT 'user',  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

Roles za mwanzo:

```
user
admin
super_admin
```

Lakini baadaye tunaweza kuwa na organization-specific roles:

```
owner
analyst
viewer
```

29. organizations

Kwa users wanaofanya kazi kama team:

SQL

```
CREATE TABLE organizations (
  id UUID PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  owner_id UUID NOT NULL REFERENCES users(id),
  plan VARCHAR(50) DEFAULT 'free',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Mfano:

```
University Research Lab
NGO Tanzania
Data Consultancy
Government Department
```

30. organization_members

SQL

```
CREATE TABLE organization_members (
  id UUID PRIMARY KEY,
  organization_id UUID REFERENCES organizations(id),
  user_id UUID REFERENCES users(id),
  role VARCHAR(50) DEFAULT 'analyst',
```

```
joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    UNIQUE(organization_id, user_id)  
);
```

Hii inaruhusu:

```
Organization  
├──  
├── Admin  
├── Analyst  
├── Analyst  
└── Viewer
```

31. projects

Project ndiyo container ya analysis.

SQL

```
CREATE TABLE projects (  
    id UUID PRIMARY KEY,  
    organization_id UUID REFERENCES organizations(id),  
    name VARCHAR(255) NOT NULL,  
    description TEXT,  
    created_by UUID REFERENCES users(id),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Mfano:

```
Project:  
Katavi Smallholder Farmers Survey 2026
```

Ndani yake:

```
Dataset  
Analysis  
Charts  
Reports
```

32. datasets

SQL

```
CREATE TABLE datasets (  
  id UUID PRIMARY KEY,  
  project_id UUID REFERENCES projects(id),  
  name VARCHAR(255) NOT NULL,  
  original_filename VARCHAR(255),  
  file_type VARCHAR(30),  
  storage_path TEXT,  
  file_size BIGINT,  
  row_count BIGINT,  
  column_count INTEGER,  
  status VARCHAR(30) DEFAULT 'PROCESSING',  
  created_by UUID REFERENCES users(id),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Status

UPLOADING
PROCESSING
READY
FAILED
ARCHIVED

33. dataset_versions

Hii ni muhimu kwa reproducibility.

SQL

```
CREATE TABLE dataset_versions (  
  id UUID PRIMARY KEY,  
  dataset_id UUID REFERENCES datasets(id),  
  version_number INTEGER NOT NULL,  
  storage_path TEXT NOT NULL,  
  row_count BIGINT,  
  file_size BIGINT,  
  operation_id UUID,  
  created_by UUID REFERENCES users(id),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  UNIQUE(dataset_id, version_number)  
);
```

Mfano:

Dataset

- |
- |— v1 Original
- |
- |— v2 Removed duplicates
- |
- |— v3 Missing values handled
- |
- |— v4 Variables recoded

34. variables

Hapa tunahifadhi metadata ya columns.

SQL

```
CREATE TABLE variables (  
  id UUID PRIMARY KEY,  
  dataset_version_id UUID REFERENCES dataset_versions(id),  
  
  name VARCHAR(255) NOT NULL,  
  label TEXT,  
  
  data_type VARCHAR(50),  
  semantic_type VARCHAR(50),  
  
  is_numeric BOOLEAN DEFAULT FALSE,  
  is_categorical BOOLEAN DEFAULT FALSE,  
  is_datetime BOOLEAN DEFAULT FALSE,  
  
  missing_count BIGINT DEFAULT 0,  
  unique_count BIGINT DEFAULT 0,  
  
  min_value DOUBLE PRECISION,  
  max_value DOUBLE PRECISION,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

`semantic_type` inaweza kuwa:

- nominal
- ordinal
- binary
- continuous
- discrete
- datetime

```
text
identifier
```

Hii itasaidia **Statistical Decision Engine**.

35. Variable Metadata

Baadaye tunaweza kuongeza:

```
measurement_level
unit
category_labels
value_labels
reference_category
role
```

Mfano:

```
education

measurement_level = ordinal

categories:
1 = Primary
2 = Secondary
3 = Diploma
4 = Degree
5 = Masters
```

Hii ni muhimu sana kwenye statistical analysis.

36. data_operations

Kila cleaning/transformation ihifadhiwe.

SQL

```
CREATE TABLE data_operations (
  id UUID PRIMARY KEY,

  dataset_id UUID REFERENCES datasets(id),
  dataset_version_id UUID REFERENCES dataset_versions(id),

  operation_type VARCHAR(50) NOT NULL,

  configuration JSONB NOT NULL,
```

```
created_by UUID REFERENCES users(id),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Mfano:

JSON

```
{
  "operation": "replace_value",
  "column": "gender",
  "from": "M",
  "to": "Male"
}
```

Au:

JSON

```
{
  "operation": "filter",
  "condition": "age >= 18"
}
```

37. Analysis

SQL

```
CREATE TABLE analyses (
  id UUID PRIMARY KEY,

  project_id UUID REFERENCES projects(id),
  dataset_version_id UUID REFERENCES dataset_versions(id),

  name VARCHAR(255),

  analysis_type VARCHAR(100),

  configuration JSONB,

  status VARCHAR(30) DEFAULT 'QUEUED',

  created_by UUID REFERENCES users(id),

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

Analysis types:

```
descriptive
frequency
correlation
chi_square
ttest
anova
linear_regression
logistic_regression
forecasting
```

38. Analysis Results

SQL

```
CREATE TABLE analysis_results (
  id UUID PRIMARY KEY,

  analysis_id UUID REFERENCES analyses(id),

  result_type VARCHAR(50),

  result_data JSONB,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Mfano:

JSON

```
{
  "sample_size": 1250,
  "mean": 523400,
  "std": 182300,
  "confidence_interval": {
    "lower": 510200,
    "upper": 536600
  }
}
```

39. Statistical Diagnostics

Ningependa hii iwe table yake.

SQL

```
CREATE TABLE analysis_diagnostics (  
  id UUID PRIMARY KEY,  
  
  analysis_id UUID REFERENCES analyses(id),  
  
  diagnostic_type VARCHAR(100),  
  
  status VARCHAR(30),  
  
  message TEXT,  
  
  details JSONB,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Mfano:

```
Diagnostic:  
Multicollinearity  
  
Status:  
WARNING  
  
Message:  
High VIF detected for farm_size.
```

Hii itasaidia AI kuelezea result kwa usahihi.

40. Charts

SQL

```
CREATE TABLE charts (  
  id UUID PRIMARY KEY,  
  
  project_id UUID REFERENCES projects(id),  
  analysis_id UUID REFERENCES analyses(id),  
  
  title VARCHAR(255),  
  
  chart_type VARCHAR(50),  
  
  configuration JSONB,
```

```
    created_by UUID REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Chart types:

```
bar
line
scatter
histogram
boxplot
violin
pie
heatmap
area
stacked_bar
```

41. Reports

SQL

```
CREATE TABLE reports (
    id UUID PRIMARY KEY,

    project_id UUID REFERENCES projects(id),

    title VARCHAR(255),

    format VARCHAR(20),

    content JSONB,

    file_path TEXT,

    status VARCHAR(30) DEFAULT 'GENERATING',

    created_by UUID REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

42. Analysis Jobs

Kwa big data, hii ni muhimu.

SQL

```
CREATE TABLE jobs (  
  id UUID PRIMARY KEY,  
  
  user_id UUID REFERENCES users(id),  
  
  project_id UUID REFERENCES projects(id),  
  
  job_type VARCHAR(100),  
  
  payload JSONB,  
  
  status VARCHAR(30),  
  
  progress INTEGER DEFAULT 0,  
  
  error_message TEXT,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  started_at TIMESTAMP,  
  
  completed_at TIMESTAMP  
);
```

Mfano:

Job:
10,000,000 row aggregation

Status:
PROCESSING

Progress:
67%

Frontend inaweza kuonyesha:

Processing dataset...

 67%

43. Indexing

Hii ni sehemu ambayo mara nyingi developers wanaipuuza.

Tutahitaji indexes kwenye:

SQL

```
CREATE INDEX idx_datasets_project
ON datasets(project_id);

CREATE INDEX idx_dataset_versions_dataset
ON dataset_versions(dataset_id);

CREATE INDEX idx_variables_version
ON variables(dataset_version_id);

CREATE INDEX idx_analysis_project
ON analyses(project_id);

CREATE INDEX idx_analysis_dataset
ON analyses(dataset_version_id);

CREATE INDEX idx_jobs_status
ON jobs(status);
```

Kwa production database, indexing itapangwa kulingana na actual query patterns.

44. Data Storage

Sasa sehemu muhimu:

Dataset rows hazikai PostgreSQL.

Tutakuwa na:

Object Storage

```
|
└─ datasets/
    │
    └─ {organization_id}/
        │
        └─ {project_id}/
            │
            └─ {dataset_id}/
                │
                └─ original/
                    │
                    └─ original.csv
                        │
```

```
├── versions/
│   ├── v1.parquet
│   ├── v2.parquet
│   └── v3.parquet
└── exports/
```

45. Data Processing Pipeline

User aki-upload CSV:

```
CSV
|
▼
Upload Service
|
▼
File Validation
|
├── File type
├── Size
├── Encoding
└── Security
|
▼
Data Profiler
|
▼
Convert → Parquet
|
▼
Object Storage
|
▼
Metadata → PostgreSQL
|
▼
Dataset READY
```

46. Security Layer

Upload yoyote ipitie:

```
File
↓
```

```
Extension validation
↓
MIME validation
↓
Size validation
↓
Malware/security scan
↓
Parsing sandbox
↓
Storage
```

Tusiruhusu user upload file halafu server i-execute content yake.

47. API Contract Example

Upload

http

```
POST /api/v1/datasets/upload
```

Response:

JSON

```
{
  "dataset_id": "uuid",
  "status": "PROCESSING"
}
```

Frontend inaweza kuanza kufuatilia:

http

```
GET /api/v1/datasets/{id}
```

Response:

JSON

```
{
  "id": "...",
  "status": "READY",
  "rows": 250000,
  "columns": 38
}
```

48. Profile API

http

```
GET /api/v1/datasets/{id}/profile
```

Response:

JSON

```
{
  "rows": 250000,
  "columns": 38,
  "missing_percentage": 4.8,
  "duplicates": 1203,
  "variables": []
}
```

49. Analysis API Example

User anaomba mean income by region.

http

```
POST /api/v1/analysis/descriptive
```

JSON

```
{
  "dataset_version_id": "...",
  "operation": "group_summary",
  "group_by": ["region"],
  "variables": ["income"],
  "statistics": [
    "count",
    "mean",
    "median",
    "std"
  ]
}
```

Response:

JSON

```
{  
  "analysis_id": "...",  
  "status": "COMPLETED"  
}
```

50. AI API

User:

"What are the main factors associated with income?"

Frontend:

http

POST /api/v1/ai/ask

JSON

```
{  
  "project_id": "...",  
  "dataset_version_id": "...",  
  "question": "What are the main factors associated with income?"  
}
```

AI response ya ndani inaweza kuwa:

JSON

```
{  
  "intent": "exploratory_relationship_analysis",  
  "target": "income",  
  "candidate_variables": [  
    "education",  
    "age",  
    "farm_size",  
    "market_access"  
  ],  
  "recommended_analyses": [  
    "correlation",  
    "group_comparison",  
    "linear_regression"  
  ]  
}
```

Kisha user anaweza:

Run recommended analysis

51. Muhimu: Reproducibility

Kila analysis ihifadhi:

```
Dataset version
+
Variables
+
Filters
+
Transformations
+
Analysis method
+
Parameters
+
Software version
+
Timestamp
```

Kwa mfano miaka miwili baadaye user anaweza kusema:

"Nataka kurudia analysis niliyofanya."

System inaweza kufanya:

Re-run Analysis

bila kuanza upya.

Hii ni muhimu sana kwa research.

52. Versioning ya Analysis

Tunaweza kuwa na:

```
Analysis #1
Linear Regression

v1
income ~ age + education

v2
income ~ age + education + farm_size
```

v3

```
income ~ age + education + farm_size + market_access
```

User anaweza kulinganisha models.

53. Statistical Output Standard

Kila analysis result itoe structured format:

1. Method
2. Sample size
3. Variables
4. Estimate
5. Standard error
6. Test statistic
7. p-value
8. Confidence interval
9. Effect size
10. Diagnostics
11. Interpretation

Hii itafanya output iwe professional kwa researchers.

54. Mfano wa Final User Experience

User ana-upload:

Farmers Survey.csv

System:

Dataset successfully analyzed.

```
250,000 observations  
38 variables  
4.8% missing values  
1,203 duplicate records
```

User:

"Does education level affect income?"

System:

Question understood.

Outcome:

Income

Grouping variable:

Education

Candidate method:

One-way ANOVA

Reason:

Continuous outcome + categorical predictor

System checks:

Sample size ✓

Missing values ✓

Outliers ⚠

Variance ⚠

Then:

ANOVA Result

$F = 12.84$

$df = 5, 1240$

$p < 0.001$

Graph:

Income distribution by education level

AI:

"The analysis found evidence of differences in mean income across education groups. The result should be interpreted alongside the group distributions and diagnostic checks."

Hapo ndiyo tunataka platform ifike.

55. Technology Stack ya mwisho kwa MVP

Layer	Technology
Frontend	Next.js + TypeScript
UI	Tailwind CSS
Charts	Apache ECharts / Plotly
Backend	Python + FastAPI
Data processing	Polars
Analytical SQL	DuckDB
Statistics	SciPy + Statsmodels
ML	Scikit-learn
Database	PostgreSQL
Dataset format	Parquet
Object storage	S3-compatible storage
Queue	Redis + Celery
Authentication	JWT + refresh tokens
AI	LLM API
Deployment	Docker
Reverse proxy	Nginx

56. Kuna uamuzi mmoja muhimu bado

Kabla ya kuanza coding, kuna **architecture decision** moja ningefanya sasa:

Tutengeneze platform kama:

A. Web-first SaaS

Browser



au

B. Web + Desktop

Browser
+
Desktop application

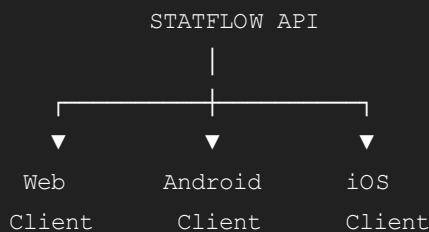
au

C. Web + Mobile App

Web
+
Android
+
iOS

Kwa product hii, ningejenga **Web-first architecture**, lakini backend API iwe tayari kutumiwa na mobile app baadaye.

Kwa hiyo:



Hii inazuia tujenge backend mara mbili.

Hatua inayofuata

Baada ya hii, hatua yenye thamani kubwa ni **kuanza MVP architecture halisi**:

1. `monorepo` structure
2. Docker setup
3. PostgreSQL
4. FastAPI
5. Next.js
6. authentication
7. dataset upload

8. CSV/XLSX parser
9. Polars + DuckDB
10. automatic profiling
11. first statistical analysis
12. first interactive chart

Kisha tutakuwa na version ya kwanza inayofanya kazi, badala ya kuendelea kubaki kwenye diagrams.

Kwa mimi, **Dataset Upload** → **Profiling** → **Cleaning** → **Descriptive Analysis** → **Visualization** ndiyo vertical slice ya kwanza tunapaswa kujenga. Baada ya hiyo kufanya kazi vizuri, tunaunganisha AI na advanced statistics juu yake.

Endelea.

57. STATFLOW MVP — ACTUAL PROJECT BLUEPRINT

Tutatumia **monorepo** kwa sababu frontend, backend na analysis engine zitahitaji kushirikiana mara kwa mara.

```
statflow/
|
├─ apps/
|   |
|   ├─ web/
|   |   └─ Next.js
|   |
|   └─ api/
|       └─ FastAPI
|
├─ packages/
|   |
|   ├─ analysis-engine/
|   ├─ data-engine/
|   ├─ shared/
|   └─ statistical-rules/
|
├─ infrastructure/
|   ├─ docker/
|   ├─ nginx/
|   └─ scripts/
|
├─ storage/
|   ├─ datasets/
|   └─ exports/
```

```
|
├─ tests/
|
├─ docker-compose.yml
├─ .env.example
├─ README.md
└─ package.json
```

58. Frontend Structure

Kwa Next.js:

```
apps/web/
|
├─ app/
|   ├── login/
|   ├── register/
|   |
|   ├── dashboard/
|   |
|   ├── projects/
|   |   └─ [projectId]/
|   |
|   ├── datasets/
|   |   └─ [datasetId]/
|   |
|   ├── analysis/
|   |
|   ├── reports/
|   |
|   └─ settings/
|
├─ components/
|   ├── Sidebar/
|   ├── Topbar/
|   ├── DatasetUploader/
|   ├── DatasetPreview/
|   ├── DataProfiler/
|   ├── CleaningPanel/
|   ├── AnalysisPanel/
|   ├── ChartBuilder/
|   └─ AIChat/
|
├─ features/
|   ├── auth/
|   ├── datasets/
|   ├── analysis/
|   ├── visualization/
|   └─ reports/
```

```
|
├─ lib/
│   ├── api.ts
│   ├── auth.ts
│   └── utils.ts
└─ types/
```

Kama ulivyopendelea kwenye projects zako nyingine, components kubwa zinaweza kuwa:

```
DataProfiler/
  DataProfiler.tsx
  DataProfiler.css
```

59. Backend Structure

```
apps/api/
├─ app/
│   ├── main.py
│   ├── api/
│   │   └── v1/
│   │       ├── auth.py
│   │       ├── projects.py
│   │       ├── datasets.py
│   │       ├── profiling.py
│   │       ├── cleaning.py
│   │       ├── analysis.py
│   │       ├── charts.py
│   │       ├── reports.py
│   │       └── ai.py
│   ├── core/
│   │   ├── config.py
│   │   ├── security.py
│   │   └── database.py
│   ├── models/
│   ├── schemas/
│   ├── services/
│   ├── repositories/
│   └── workers/
```

60. Data Engine

Hii ndiyo engine inayoshughulika na datasets.

```
packages/data-engine/  
|  
├─ readers/  
|   ├── csv_reader.py  
|   ├── excel_reader.py  
|   ├── json_reader.py  
|   └─ parquet_reader.py  
|  
├─ profiling/  
|   ├── profiler.py  
|   ├── column_profiler.py  
|   ├── missing.py  
|   ├── duplicates.py  
|   └─ outliers.py  
|  
├─ cleaning/  
|   ├── missing.py  
|   ├── duplicates.py  
|   ├── recoding.py  
|   └─ types.py  
|  
├─ transformations/  
|   ├── filter.py  
|   ├── mutate.py  
|   ├── groupby.py  
|   ├── join.py  
|   └─ pivot.py  
|  
└─ storage/  
    ├── parquet.py  
    └─ metadata.py
```

61. Analysis Engine

```
packages/analysis-engine/  
|  
├─ core/  
|   ├── base.py  
|   ├── registry.py  
|   └─ result.py
```

```
|
|└─ descriptive/
|   |└─ summary.py
|   |└─ frequency.py
|   └─ distribution.py
|
|└─ inferential/
|   |└─ ttest.py
|   |└─ anova.py
|   |└─ chi_square.py
|   └─ correlation.py
|
|└─ regression/
|   |└─ linear.py
|   └─ logistic.py
|
|└─ diagnostics/
|   |└─ normality.py
|   |└─ outliers.py
|   |└─ multicollinearity.py
|   └─ heteroscedasticity.py
```

62. Analysis Registry

Hatuta-hardcode kila analysis kwenye API.

Tutakuwa na registry:

Python

Run

```
ANALYSIS_REGISTRY = {
    "summary": SummaryAnalyzer,
    "frequency": FrequencyAnalyzer,
    "correlation": CorrelationAnalyzer,
    "ttest": TTestAnalyzer,
    "chi_square": ChiSquareAnalyzer,
    "anova": AnovaAnalyzer,
    "linear_regression": LinearRegressionAnalyzer,
}
```

Hivyo tukiongeza method mpya:

```
logistic_regression
```

tunai-register tu.

63. Standard Analysis Contract

Kila analyzer itafuata structure moja:

Python

Run

```
class Analyzer:

    def validate(self, dataset, config):
        ...

    def run(self, dataset, config):
        ...

    def diagnostics(self, result):
        ...

    def summarize(self, result):
        ...
```

Hii ni muhimu sana.

Frontend haitahitaji kujua internal details za ANOVA, regression au chi-square.

Itapokea standard result.

64. Standard Result Object

Mfano:

JSON

```
{
  "analysis_type": "ttest",
  "status": "completed",

  "sample_size": 2500,

  "estimate": {
    "group_a_mean": 523000,
    "group_b_mean": 481000
  },

  "test": {
    "statistic": 3.82,
    "p_value": 0.00014
  },
}
```

```
"confidence_interval": {
  "lower": 20400,
  "upper": 63600
},

"effect_size": {
  "name": "cohens_d",
  "value": 0.21
},

"diagnostics": [],

"warnings": []
}
```

65. Data Profiling Engine

User aki-upload data, profiler ifanye automatically:

Dataset

- |
- ├─ Shape
- ├─ Data types
- ├─ Missing
- ├─ Duplicates
- ├─ Unique values
- ├─ Numeric distributions
- ├─ Categorical distributions
- ├─ Date detection
- ├─ ID detection
- └─ Potential errors

66. Automatic Type Detection

Mfano column:

age

21
24
36
45
28

System:

Physical type:
integer

Semantic type:
continuous

Lakini:

gender

Male

Female

Male

Female

System:

Physical type:
string

Semantic type:
nominal

Na:

education

Primary

Secondary

Diploma

Degree

system inaweza kupendekeza:

ordinal

Lakini **isifanye assumption** bila kumpa user nafasi ya kurekebisha.

67. Missing Data Intelligence

Tutaonyesha:

age

Missing: 2.1%

```
income
Missing: 8.7%

education
Missing: 0.4%
```

Lakini hatutafanya automatic imputation bila user consent.

System inaweza kupendekeza:

```
income:

Recommended:
Median imputation

Alternative:
Multiple imputation
Complete-case analysis
```

68. Data Cleaning History

Frontend iwe na panel:

```
DATA OPERATIONS

✓ Removed 1,203 duplicates
✓ Replaced M → Male
✓ Replaced F → Female
✓ Created income_group
✓ Filtered age >= 18

[Undo]
```

Hii itakuwa **data transformation pipeline**.

69. Transformation Pipeline

Badala ya kubadilisha file moja kwa moja:

```
Original Dataset
  ↓
Operation 1
  ↓
Operation 2
  ↓
```

Operation 3

↓

Current Version

Mfano:

```
v1
↓
remove_duplicates
↓
v2
↓
recode_gender
↓
v3
↓
create_income_group
↓
v4
```

Hii inasaidia:

- reproducibility
- undo
- audit
- research
- collaboration

70. Interactive Data Table

Data table iwe na features:

```
Search
Filter
Sort
Hide column
Show column
Rename
Format
Statistics
```

User akiclick column:

```
income
_____
Type: Numeric
Missing: 8.7%
Mean: 523,000
```

Median: 410,000
SD: 182,300
Min: 20,000
Max: 4,800,000

[Analyze]
[Transform]
[Visualize]

71. First MVP Analysis

Tunaanza na **Descriptive Statistics**.

User:

Select variables:

- ☒ age
- ☒ income
- ☒ farm_size

Click:

Analyze

System:

Variable	N	Mean	Median	SD
age	24920	42.6	41	12.8
income	23100	523400	410000	182300
farm_size	24800	2.84	2.1	2.31

72. First Visualization

Kutoka hapo:

Visualize

Variable:
income

Chart:
Histogram

Frontend inapata data/configuration na chart library ndiyo inachora.

Hatutaki backend kutuma picha ya chart kwa kila interaction.

Badala yake:

```
Backend
  ↓
Chart specification
  ↓
Frontend
  ↓
Interactive chart
```

Hii itakuwa fast.

73. Chart Specification

Mfano:

```
JSON

{
  "type": "scatter",
  "x": "farm_size",
  "y": "income",
  "title": "Farm Size and Household Income",
  "encoding": {
    "x": {
      "type": "quantitative"
    },
    "y": {
      "type": "quantitative"
    }
  }
}
```

Frontend inachora.

74. AI "Ask Data"

Tutafanya UI kama:

```
| Ask about your data |
|                       |
| "What are the main factors associated |
```

```
| with household income?"
```

```
| [Ask StatFlow]
```

AI itaweza kutoa:

```
I identified household income as the  
outcome variable.
```

```
Potential explanatory variables include:
```

- education
- age
- farm size
- market participation

```
Would you like me to run a regression?
```

75. AI lazima ijue dataset context

Prompt architecture inaweza kuwa:

```
SYSTEM
```

```
You are StatFlow statistical assistant.
```

```
DATASET METADATA
```

```
...
```

```
VARIABLES
```

```
...
```

```
USER QUESTION
```

```
...
```

```
AVAILABLE ANALYSIS METHODS
```

```
...
```

```
STATISTICAL RULES
```

```
...
```

Lakini **data yote** hatuitumi kwa LLM.

Tunatuma:

- metadata
- relevant summaries
- selected results

Hii inapunguza:

- cost
- latency
- privacy risk
- token usage

76. Big Data + AI

Kwa dataset yenye:

10 million rows

user akiuliza:

"What is the average income by region?"

Hatufanyi:

10m rows → LLM

Tunafanya:

```
10m rows
  ↓
DuckDB/Polars
  ↓
Aggregation
  ↓
Small result table
  ↓
AI
```

AI inaweza kupewa:

Region | N | Mean

badala ya rows zote milioni 10.

Hii ni **fundamental architecture decision**.

77. Performance Strategy

Tutatumia:

Lazy execution

Polars/DuckDB inapowezekana.

Column projection

Ikiwa analysis inahitaji:

```
income
education
age
```

hatupakii columns zote 200.

Predicate pushdown

Filter ifanyike karibu na storage layer.

Caching

Mfano analysis ile ile:

```
Dataset v4
+
same query
```

result inaweza kutolewa kwenye cache.

Background jobs

Analysis kubwa isiblock API.

78. Frontend Performance

Data table yenye:

```
10,000,000 rows
```

hatuwezi ku-render rows zote.

Tutatumia:

virtualized table

na server-side:

pagination
filtering
sorting

User anaona:

Rows 1-100

Iakini system inaendelea kuwa na dataset nzima backend.

79. MVP Development Order

Sasa development order iwe hii:

Sprint 1

Project setup
Docker
PostgreSQL
FastAPI
Next.js
Authentication

Sprint 2

Dataset upload
CSV
XLSX
Storage
Dataset metadata

Sprint 3

Profiling
Variable detection
Missing values
Duplicates
Data preview

Sprint 4

Cleaning
Filtering

Recoding
Transformation
Versioning

Sprint 5

Descriptive statistics
Frequency
Correlation

Sprint 6

Charts
Dashboard
Interactive visualization

Sprint 7

t-test
Chi-square
ANOVA
Regression

Sprint 8

AI Assistant
Ask Data
Analysis recommendations
Result explanation

80. MVP Definition of Done

Sitaita "MVP complete" mpaka user aweze kufanya hii bila kuandika code:

```
1. Register
   ↓
2. Create project
   ↓
3. Upload CSV/XLSX
   ↓
4. System profiles dataset
   ↓
5. Inspect variables
   ↓
```

```
6. Clean data
  ↓
7. Create transformed version
  ↓
8. Run statistics
  ↓
9. Generate chart
  ↓
10. Ask AI about result
  ↓
11. Export result
```

Hii ndiyo **first usable product**.

81. Lakini kuna feature moja ningependekeza tuijenge tofauti

Research Mode

Kwa sababu platform hii inaweza kutumiwa sana na students/researchers.

User anaweza kuweka:

```
Research Question
Objectives
Hypotheses
Variables
Study Design
```

Mfano:

Objective: Determine relationship between post-harvest losses and formal market participation.

System inaweza ku-link:

```
Objective
  ↓
Variables
  ↓
Hypothesis
  ↓
Statistical Method
  ↓
Analysis
  ↓
Result
```



Hii ni powerful sana kwa academic users.

82. Survey Mode

Na kutokana na background yako ya Official Statistics, baadaye:

```
Standard Mode
Research Mode
Survey Mode
Business Analytics Mode
```

Survey Mode

```
Sampling design
Weights
Strata
PSU
Weighted estimates
SE
CI
Design effect
```

Hii inaweza kuifanya platform iwe useful hata kwa professional survey analysis.

83. Product Vision

Kwa hiyo hatujengi:

"Another Excel."

Wala:

"Another SPSS clone."

Tunajenga:

An intelligent statistical workspace

ambayo inachanganya:

```
Data Preparation
+
Statistical Analysis
```

+
Visualization
+
AI Assistance
+
Research Workflow
+
Big Data Processing

Hatua inayofuata: tuanze BUILD

Kwa sasa architecture yetu imeshakaa. Hatua inayofuata ninayopendekeza si kuongeza diagrams zaidi.

Tuanze **MVP-1 codebase** kwa mpangilio huu:

STEP 1
Monorepo + Docker

STEP 2
PostgreSQL + migrations

STEP 3
FastAPI backend

STEP 4
Next.js frontend

STEP 5
Auth

STEP 6
Dataset upload

STEP 7
Polars/DuckDB data engine

STEP 8
Automatic profiling

STEP 9
Interactive data table

STEP 10
First descriptive analysis

STEP 11
First charts

Baada ya hatua hizo, tutakuwa na **StatFlow v0.1 inayofanya kazi**, ndipo tujenge t-test, ANOVA, regression, AI na Survey Mode juu ya foundation hiyo.

Endelea

Tumefika kwenye **MVP-1 ya kwanza ya StatFlow**. Nimeandaa starter project yenye:

- **Next.js + TypeScript** frontend
- **FastAPI + Python** backend
- **PostgreSQL** kupitia Docker
- CSV/XLSX upload
- Automatic dataset profiling
- Row/column count
- Missing values
- Unique values
- Variable/data-type detection
- Dataset conversion kwenda **Parquet**
- API documentation kupitia [/docs](#)
- Docker Compose ya ku-run mfumo mzima

Muundo wa sasa

```
statflow-mvp1/
├─ apps/
│   └─ web/
│       └─ app/
│           ├── layout.tsx
│           ├── page.tsx
│           └─ globals.css
│       └─ Dockerfile
│       └─ package.json
│       └─ tsconfig.json
│   └─ api/
│       ├── app/
│       │   ├── main.py
│       │   └─ __init__.py
│       ├── Dockerfile
│       └─ requirements.txt
├─ storage/
└─ datasets/
```

```
| └─ exports/  
└─ docker-compose.yml  
└─ .env.example  
└─ README.md
```

Pakua project

Download StatFlow MVP-1 ZIP

Baada ya ku-extract, command ya kuanzia ni:

Bash

```
docker compose up --build
```

Kisha:

```
Frontend:  http://localhost:3000  
Backend:   http://localhost:8000  
API Docs:  http://localhost:8000/docs
```

Hatua inayofuata

Sasa sitaki turuke moja kwa moja kwenye AI. **Hatua inayofuata ni kujenga Data Engine vizuri**, kwa sababu ndiyo moyo wa StatFlow:

```
Upload  
  ↓  
Profiling  
  ↓  
Data Table  
  ↓  
Clean  
  ↓  
Transform  
  ↓  
Versioning  
  ↓  
Analysis
```

Tutajenga **Dataset Workspace** yenye tabs:

Overview | Data | Clean | Transform | Analyze | Visualize | AI | Report

na Data Table iweze kufanya filtering, sorting, search, column statistics, missing-value handling na pagination bila kujaribu kupakia dataset kubwa yote browser.



Sawa. Sasa tunaingia kwenye **MVP-2: Dataset Workspace + Data Engine**. Hii ndiyo sehemu muhimu zaidi kabla ya statistics na AI.

83. Dataset Workspace

Baada ya user ku-upload dataset, hatumpeleki kwenye dashboard tu. Tunampeleka kwenye workspace kama:

StatFlow / Projects / Farmers Survey					
Overview	Data	Clean	Transform	Analyze	Visualize AI
Dataset: Farmers_Survey.xlsx					
250,430 rows • 42 variables • 4.8% missing					
[Search variables]			[Export]		
Variable	Type	Missing	Unique	Statistics	
age	Numeric	2.1%	72	→	
gender	Categorical	0.2%	3	→	
income	Numeric	8.7%	4,320	→	

84. Overview Tab

User akifungua dataset anaona:

Dataset health

Rows	250,430
Variables	42
Missing values	4.8%
Duplicate rows	1,203
Potential errors	87

Na health indicator:

Data Quality



89%

Lakini score hii iwe **explainable**, si namba ya kubuni tu.

Mfano:

Missing values	-5
Duplicates	-3
Invalid values	-2
Type inconsistencies	-1

85. Data Tab

Tutahitaji data-grid yenye uwezo wa:

- Search
- Filter
- Sort
- Resize columns
- Hide columns
- Pin columns
- Select rows
- Column menu

Lakini muhimu:

Browser haitapokea rows 250,000 zote.

Badala yake:

```
Browser
  ↓
GET /datasets/{id}/rows
  ↓
Backend
  ↓
DuckDB/Parquet
  ↓
Only requested rows
```

Mfano:

http

```
GET /api/v1/datasets/123/rows?offset=0&limit=100
```

Next:

http

```
GET /api/v1/datasets/123/rows?offset=100&limit=100
```

86. Filtering

User akisema:

```
age >= 18
AND
income > 500000
```

Frontend isitume Python code.

Itume structured query:

```
JSON

{
  "filters": [
    {
      "column": "age",
      "operator": ">=",
      "value": 18
    },
    {
      "column": "income",
      "operator": ">",
      "value": 500000
    }
  ]
}
```

Backend ndiyo itengeneze safe query.

87. Transformation Engine

Tutakuwa na operations kama:

Create variable

```
income_per_person = income / household_size
```

Recode

gender:

M → Male

F → Female

Categorize

age

18-30 → Young

31-50 → Middle

51+ → Older

Log transformation

```
log_income = log(income)
```

Standardization

```
z_income = (income - mean) / std
```

88. Transformation UI

User akichagua:

Transform → Create Variable

ataona:

Variable name

[income_per_person]

Formula

[income / household_size]

Preview

income	household_size	result
500000	5	100000
800000	4	200000

[Cancel] [Apply]

Kabla ya apply, system ifanye validation.

89. Safe Formula Engine

Hatutaki:

```
eval(user_input)
```

kabisa.

Tutakuwa na expression parser:

```
column
number
+
-
*
/
%
()
functions
```

Allowed functions:

```
log()
sqrt()
abs()
round()
floor()
ceil()
```

Na validation ya column names.

90. Cleaning Tab

UI:

CLEAN DATA

Missing Values

income	8.7%
education	0.4%

[Remove rows]
[Fill with mean]
[Fill with median]
[Fill with mode]
[Forward fill]
[Custom value]

Duplicates

1,203 duplicate rows

[Review]
[Remove duplicates]

Muhimu: **hakuna destructive operation bila confirmation.**

91. Before / After Preview

Mfano user anataka remove duplicates:

Before

250,430 rows

↓

Remove duplicates

↓

After

249,227 rows

System iulize:

1,203 duplicate records will be removed. Create new dataset version?

[Cancel] [Create Version]

92. Versioning

Baada ya operation:

Dataset Versions

v1	Original
	250,430 rows
v2	Removed duplicates
	249,227 rows
v3	Missing income handled
	249,227 rows
v4	Created income_per_person
	249,227 rows

User anaweza kurudi v1 bila kupoteza original.

93. Operation Log

Kila operation ihifadhi:

JSON

```
{
  "operation": "remove_duplicates",
  "dataset_version": "v1",
  "output_version": "v2",
  "columns": ["household_id"],
  "rows_removed": 1203,
  "created_at": "..."}
}
```

Hii ndiyo msingi wa **audit trail**.

94. Data Quality Engine

Profiler iwe na rules.

Mfano:

Numeric

```
missing
infinite
```

negative where invalid
extreme values

Categorical

missing
unexpected categories
spelling inconsistencies

Mfano:

Gender values:

Male	4,201
Female	4,103
M	2,003
F	1,981
male	22
female	18

System inaweza kusema:

Potential category inconsistency detected.

Badala ya kubadilisha automatically.

95. Outlier Detection

Tutaanza na:

IQR
Z-score
Modified Z-score

Mfano:

income

Potential outliers: 213

User akifungua:

Method: IQR

Q1 = 250,000
Q3 = 650,000

IQR = 400,000

Lower bound = -350,000

Upper bound = 1,250,000

Then:

[Review records]

96. Muhimu: Outlier ≠ Error

StatFlow isifanye:

outlier → delete

Automatic.

Iweke:

Potential outlier

kwa sababu observation inaweza kuwa halali.

Hii ni muhimu sana kwenye statistical analysis.

97. Variable Inspector

User akiclick `income`:

INCOME

Type

Numeric

Measurement

Continuous

Rows

249,227

Missing

8.7%

```
Unique
4,320

Minimum
20,000

Maximum
4,800,000

Mean
523,400

Median
410,000

Std. deviation
182,300
```

Chini:

```
Distribution
[Histogram]

Missingness
[Chart]

[Analyze variable]
[Transform]
[Create chart]
```

98. Statistical Engine inianza hapa

Tutengeneze first endpoint:

```
http
```

```
POST /api/v1/analysis/descriptive
```

Request:

```
JSON
```

```
{
  "dataset_version_id": "...",
  "variables": [
    "age",
    "income",
    "farm_size"
```

```
],
"statistics": [
  "count",
  "missing",
  "mean",
  "median",
  "std",
  "min",
  "max"
]
}
```

Response:

JSON

```
{
  "analysis_id": "...",
  "results": [
    {
      "variable": "age",
      "count": 249210,
      "missing": 17,
      "mean": 42.61,
      "median": 41,
      "std": 12.82,
      "min": 18,
      "max": 91
    }
  ]
}
```

99. Frequency Analysis

Kwa categorical:

JSON

```
{
  "analysis_type": "frequency",
  "variable": "gender"
}
```

Result:

Male	52.4%
Female	47.1%
Other	0.5%

Na chart:

Gender Distribution



100. Cross-tabulation

Tutafuata na:

```
gender × market_participation
```

Result:

	Formal	Informal	None
Male	42%	37%	21%
Female	31%	43%	26%

Then user anaweza:

Run Chi-square

StatFlow itself itaelewa kwamba:

```
2 categorical variables
    ↓
Contingency table
    ↓
Chi-square candidate
```

Hii ndiyo mwanzo wa **Analysis Recommendation Engine**.

101. Analysis Recommendation Engine

Badala ya user kujua statistics zote, mfumo utaangalia:

```
Outcome
Predictor
Variable types
```

Number of groups
Study design
Sample size
Missingness

Mfano:

Outcome:
income → continuous

Predictor:
gender → binary categorical

System:

Possible analyses:

- ✓ Independent samples t-test
- ✓ Mann-Whitney U

[Compare groups]

Mfano mwingine:

Outcome:
income → continuous

Predictor:
education → 5 categories

System:

Possible analyses:

- ✓ One-way ANOVA
- ✓ Kruskal-Wallis

[Compare groups]

102. Hapa ndipo "AI" inanza kuwa useful

AI isiwe calculator.

Architecture iwe:

```
User question
↓
Intent detection
↓
Dataset metadata
↓
Statistical rules
↓
Candidate methods
↓
Diagnostics
↓
Statistical engine
↓
Verified result
↓
AI explanation
```

AI haijitungii p-value.

Engine ndiyo inahesabu.

AI inaelezea result iliyotengenezwa na engine.

103. Example

User:

"Je, elimu ina uhusiano na kipato?"

AI/intent layer:

```
Target:
income

Predictor:
education

Types:
continuous × categorical

Candidate:
ANOVA
```

Engine:

ANOVA

$F = 12.84$

$p < 0.001$

Diagnostics:

Normality: warning

Variance homogeneity: warning

AI response:

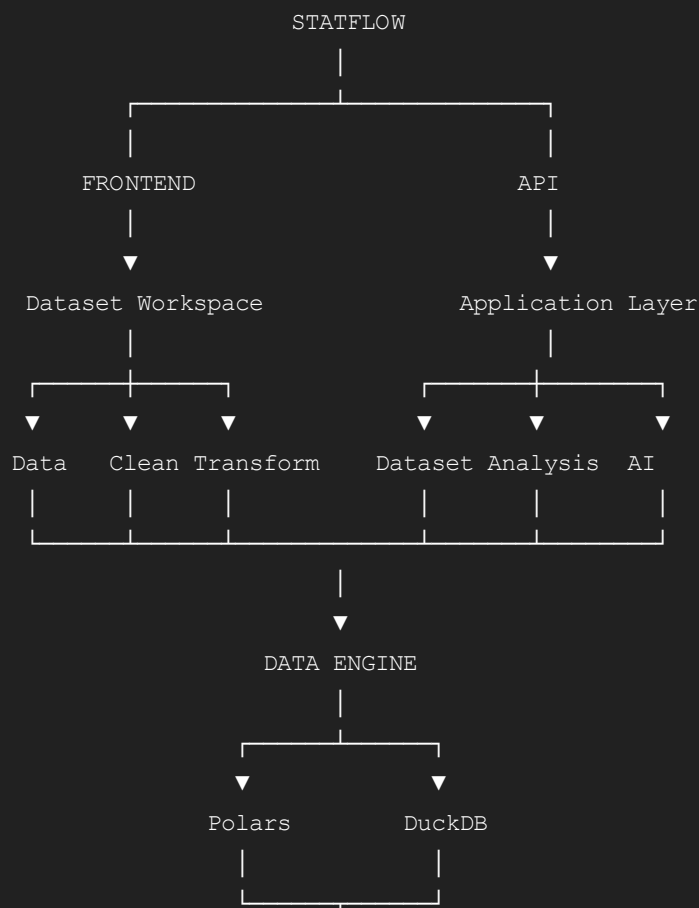
Kuna tofauti ya wastani wa kipato kati ya makundi ya elimu katika sampuli hii. Hata hivyo, assumptions za ANOVA zinapaswa kuangaliwa; mfumo pia unapendekeza Kruskal–Wallis kama robustness check.

Hii ni tofauti kubwa na chatbot anayesema tu:

"Education has a significant effect on income."

104. MVP-2 Architecture

Kwa hiyo sasa architecture yetu inakuwa:



105. Kitu muhimu kuhusu Big Data

Tukifikia datasets kubwa sana, architecture inaweza kuendelea hivi:

```
1 MB
  ↓
Polars in-memory

500 MB
  ↓
Polars / DuckDB

5 GB
  ↓
DuckDB + Parquet

50 GB+
  ↓
Distributed processing option

100 GB+
  ↓
Cluster / cloud data engine
```

Hatutaanza na distributed infrastructure mapema kwa sababu itaongeza complexity bila sababu.

Tutajenga abstraction:

Python

Run

DataEngine

ili baadaye engine ibadilike bila kuvunja frontend/API.

106. Next Architecture Layer

Baada ya MVP-2, tunajenga **Statistical Intelligence Layer**:

```
statistical-rules/
|
├─ variable_types/
├─ assumptions/
```

```
├─ tests/
├─ effect_sizes/
├─ confidence_intervals/
├─ model_selection/
└─ recommendations/
```

Mfano:

Python

Run

```
recommend(
    outcome="income",
    predictor="education",
    design="cross_sectional"
)
```

returns:

JSON

```
{
  "primary": ["anova"],
  "alternative": ["kruskal_wallis"],
  "reason": "continuous outcome + categorical predictor"
}
```

107. Product sasa inaanza kuwa tofauti

User hatalazimika kuuliza:

"Which statistical test should I use?"

Anaweza kusema:

"Nataka kujua kama elimu inahusiana na kipato."

StatFlow inamsaidia kufika:

```
Question
↓
Variables
↓
Method
↓
Assumptions
↓
Analysis
↓
```

Visualization
↓
Interpretation
↓
Report

Hii ndiyo feature ambayo ningependa iwe **core identity** ya StatFlow.

Hatua yetu inayofuata

Sasa tunaweza kuingia **MVP-3: Statistical Engine**, ambapo tutajenga kwa mpangilio:

1. Descriptive Statistics
2. Frequency Tables
3. Cross-tabulation
4. Correlation
5. t-test
6. Chi-square
7. ANOVA
8. Linear Regression
9. Logistic Regression
10. Assumption Diagnostics
11. Effect Sizes
12. Confidence Intervals
13. Analysis Recommendation Engine

Na baada yake **Visualization Engine + AI Assistant + automatic report generation**.

Hapo ndipo tutaanza kuifanya StatFlow isiwe tu data-cleaning application, bali **full statistical analysis platform**.

Endelea

Tumekamilisha **MVP-3: Statistical Engine foundation**.

Sasa StatFlow inaweza kufanya

1. Descriptive statistics

- Count
- Missing
- Mean
- Median

- Standard deviation
- Minimum
- Maximum

2. Frequency analysis

- Frequency
- Percentage
- Missing values

3. Correlation

- Pearson
- Spearman
- Correlation coefficient
- p-value

4. Independent samples t-test

- Groups mbili
- Group means
- Mean difference
- t-statistic
- p-value
- Welch's t-test

5. Chi-square

- Contingency table
- Chi-square statistic
- p-value
- Degrees of freedom
- Cramér's V

6. Common result structure

- Sample size
- Variables
- Estimates
- Test statistics
- p-values
- Effect sizes
- Warnings/diagnostics

Download

Download StatFlow MVP-3 Statistical Engine

Jambo muhimu nililoweka kwenye architecture

Hatutaandika kila analysis kivyake bila mfumo. Kuna:

```
Analysis Request
  ↓
Validation
  ↓
Analysis Service
  ↓
Specific Analyzer
  ↓
Standard Result
  ↓
Visualization / AI / Report
```

Kwa hiyo baadaye tukiweka:

```
ANOVA
Regression
Logistic Regression
Mann-Whitney
Kruskal-Wallis
PCA
Time Series
Survey Analysis
```

hazitavunja architecture iliyopo.

Hatua inayofuata

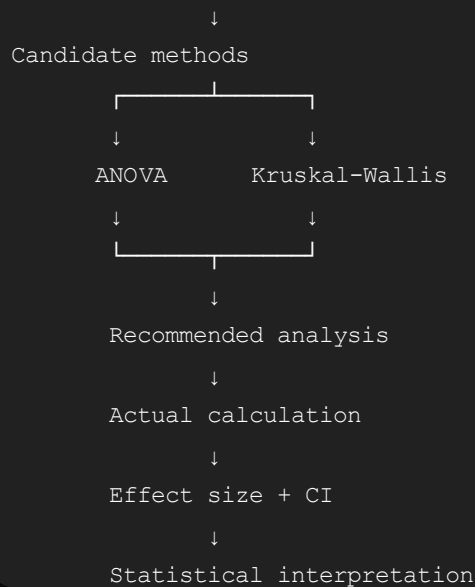
Sasa ningependekeza tuingie **MVP-4: Statistical Validation & Recommendation Engine**.

Hapo tutajenga kitu muhimu sana:

```
User:
"Je education ina relationship na income?"

  ↓

Variable detection
  ↓
education = categorical
income = continuous
  ↓
Sample-size check
  ↓
Missing-value check
  ↓
Assumption checks
```



Hii ndiyo layer itakayofanya **StatFlow** iwe **intelligent statistical platform**, badala ya kuwa collection ya statistical calculators. ☐

Endelea

Tumekamilisha **MVP-4: Statistical Validation & Recommendation Engine**.

Hii ni hatua muhimu sana kwa StatFlow kwa sababu sasa mfumo hauishii kwenye “run test”; unaanza **kuamua analysis ipi inafaa kulingana na aina ya variables na data iliyopo**.

Vipengele vilivyoongezwa

1. Automatic variable typing

Mfumo unatofautisha:

```
continuous
discrete
binary
categorical
datetime
text
```

2. Data validation

Kabla ya analysis:

```
Variable exists?
  ↓
Correct variable type?
  ↓
Enough observations?
  ↓
Missingness acceptable?
  ↓
Assumptions?
  ↓
Analysis
```

3. Missingness checks

Mfano:

```
income
Missing = 13.4%

⚠ Warning:
Missingness is above the configured threshold.
```

4. Sample-size checks

Mfumo hautaruhusu analysis kama observations hazitoshelezi.

5. Assumption diagnostics foundation

Tumeweka msingi wa:

- Shapiro-Wilk normality test
- Levene's test for equality of variances

6. Statistical Recommendation Engine

Mfano:

```
Outcome: income
Predictor: education

Detected:
income    → continuous
education → categorical

Possible methods:
_____
1. One-way ANOVA
2. Kruskal-Wallis
```

Au:

income × gender

1. Independent t-test
2. Mann-Whitney U

Au:

income × farm_size

1. Pearson correlation
2. Spearman correlation
3. Linear regression

Au:

gender × market_participation

1. Chi-square
2. Fisher's exact test

Download

Download StatFlow MVP-4

Sasa hatua inayofuata ni kubwa zaidi

Tunapaswa kujenga **MVP-5: ANOVA + Regression + Diagnostics + Effect Sizes + Confidence Intervals**.

Hapo mfumo utanza kutoa result kama:

ONE-WAY ANOVA

$F(4, 892) = 12.84$
 $p < 0.001$

Effect size
 $\eta^2 = 0.054$

95% CI
...

Diagnostics
✓ Sample size

△ Normality
✓ Independence
△ Homogeneity of variance

Alternative:
Kruskal-Wallis

Na kwa regression:

Linear Regression

R² 0.427
Adjusted R² 0.421

Variable	Coef.	SE	p-value
Education	52,430	8,210	<0.001
Farm size	17,820	3,420	<0.001
Age	1,210	520	0.020

Kisha tutaunganisha hii na **Visualization Engine**, ambapo kila analysis inaweza kutoa automatically:

table → **chart** → **statistical result** → **interpretation** → **report**. ☐

Endelea

Tumekamilisha **MVP-5: Statistical Models & Diagnostics**.

Sasa StatFlow imeanza kuwa statistical analysis platform halisi, si data profiler pekee.

Tumeongeza

1. One-way ANOVA

Mfumo unaweza kutoa:

F-statistic
p-value
df between
df within
group means
sample size

Pamoja na:

η^2 (Eta Squared)

kwa **effect size**.

2. Confidence Intervals

Kwa group means:

Mean

95% CI Lower

95% CI Upper

Hii itakuwa muhimu sana baadaye kwenye automatic reports.

3. ANOVA Diagnostics

Tumeweka msingi wa:

Levene's test

kuangalia homogeneity of variances.

Mfano:

Variance Homogeneity

Levene p = 0.083

Status: Passed

au:

Levene p < 0.001

Status: Warning

Consider:

Kruskal-Wallis

Welch ANOVA

4. Multiple Linear Regression

Sasa user anaweza kuwa na:

Outcome:
income

Predictors:
education
age
farm_size
experience

na kupata:

R²
Adjusted R²
AIC
BIC

pamoja na coefficient table:

Variable	Coef.	SE	t	p-value	95% CI
Education	β	SE	t	p	CI
Age	β	SE	t	p	CI
Farm size	β	SE	t	p	CI

5. Regression diagnostics foundation

Tumeanza kuweka:

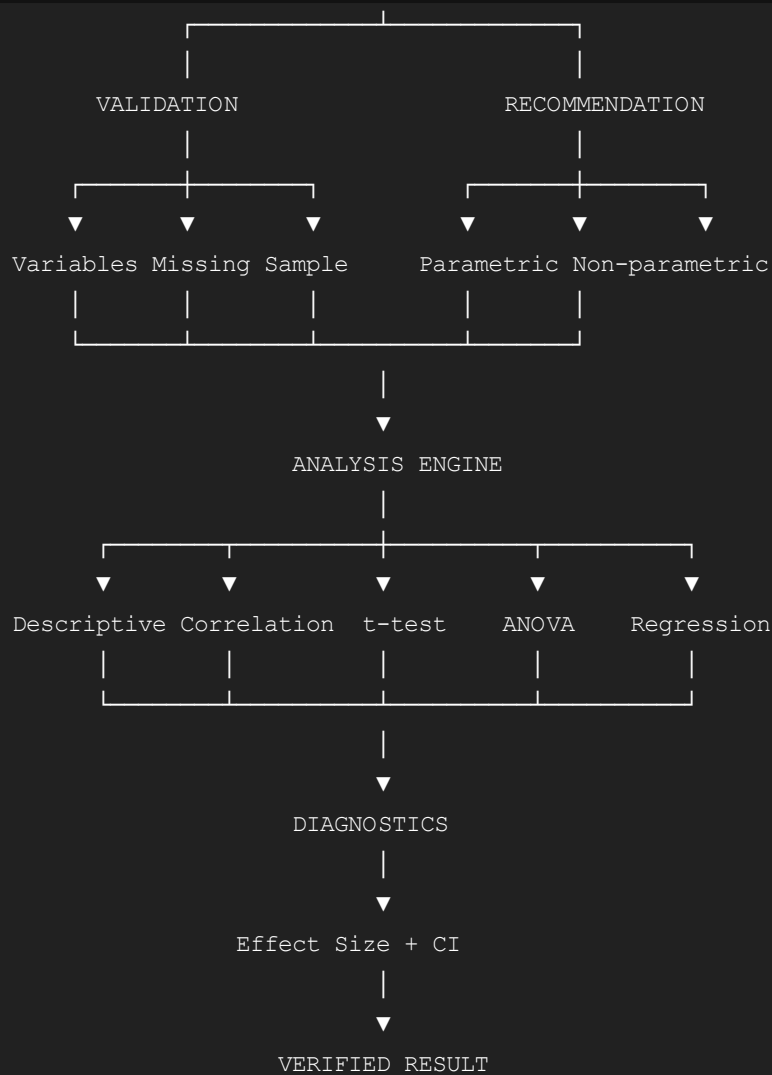
Residual normality
Durbin-Watson
Residual variance

Hii itapanuliwa kwenye hatua inayofuata.

Download

[Download StatFlow MVP-5](#)

Sasa architecture ya statistics iko hivi



Hatua inayofuata: MVP-6

Sasa tunahitaji kujenga **Visualization Engine**.

Tutafanya mfumo ambao hauishii kusema:

Mean income = 523,400

bali automatically unaonyesha:

```
Analysis
  ↓
Result
  ↓
Best chart suggestion
  ↓
Interactive chart
  ↓
Download PNG/SVG
  ↓
Add to Report
```

Na chart engine itajua tofauti kati ya:

```
Categorical → Bar / Donut
Numeric distribution → Histogram / Boxplot
Numeric × Numeric → Scatter
Time → Line
Group comparison → Boxplot / Bar + CI
Correlation → Scatter + trend line
Regression → Actual vs Predicted / Residual plots
```

Hapo ndipo tutaunganisha **ECharts/Plotly + statistical results**, kabla ya kuingia kwenye AI Assistant. ☐

Endelea

Worked for 1m 27s

Tumekamilisha **MVP-6: Visualization Engine** na nimeunganisha MVP-1 → MVP-5 kuwa project moja.

Kilichoongezwa

- **Chart Recommendation Engine**
 - Numeric × Numeric → Scatter
 - Numeric distribution → Histogram
 - Categorical → Bar
 - Time series → Line
 - Group comparison / ANOVA → Boxplot foundation
- **Interactive charts** kwa ECharts
- Visualization Builder yenye:
 - X variable
 - Y variable
 - Analysis context
 -
- Chart Data API
- Consolidated backend:
 - Upload CSV/XLSX
 - Profiling
 - Statistical analyses
 - Validation
 - ANOVA

- Linear regression
- Visualization
- Repository sasa ni snapshot moja badala ya ZIP zinazojitegemea.

Download MVP-6: [statflow-mvp6-visualization.zip](#)

Kumbuka: nimehakiki Python syntax, lakini Docker haipo kwenye mazingira haya, hivyo sikuweza kufanya `docker compose build` hapa.

Hatua inayofuata: MVP-7

Ningependekeza sasa tuingie kwenye **Professional Data Workspace**, ambapo tutafanya:

Upload → Data Table → Clean → Transform → Analyze → Visualize

hasa:

1. Data table yenye pagination
2. Search/filter/sort
3. Rename columns
4. Change data types
5. Missing-value handling
6. Remove duplicates
7. Recode variables
8. Create calculated variables
9. Filter rows
10. Group & aggregate
11. Dataset versioning / Undo

Hii ndiyo hatua itakayoanza kuifanya StatFlow iwe **actual data-analysis application**, si dashboard yenye charts tu.

Endelea

Worked for 1m 31s

Tumefika **MVP-7: Professional Data Workspace**. Nimeunda snapshot mpya iliyounganisha MVP-6 na kuongeza data manipulation/cleaning ya msingi.

MVP-7 sasa ina

Data

- Server-side pagination

- Search kwenye columns zote
- Sort ascending/descending
- Data table
- Version selector

Clean

- Remove duplicates
- Missing values:
 - Mean
 - Median
 - Mode
 - Zero
 - Drop rows
- Rename columns
- Type casting foundation

Transform

- Filter rows:
 - 
 - 
 - 
 - 
 - 
 - 
 - contains
- Calculated columns:
 - 
 - 
 - 
 - 
- Select columns

Dataset Versioning

Mfano:

```
Original
  ↓
Version 1
  ↓ remove duplicates
Version 2
  ↓ fill missing values
Version 3
```

↓ create calculated variable
Version 4

Original data haibadilishwi. Kila operation inaingia kwenye `operations.jsonl`, hivyo baadaye tunaweza kujenga **Undo/Redo + reproducibility** vizuri.

Download

Download StatFlow MVP-7

API documentation iko kwenye:

`http://localhost:8000/docs`

Baada ya MVP-7, hatua inayofuata ninayoona muhimu ni **MVP-8: Advanced Statistical Analysis Workspace** — ambapo tutaunganisha Data Workspace na panel ya kuchagua analysis, assumptions/diagnostics, results tables, effect sizes, confidence intervals na interpretation.

Endelea

Worked for 1m 2s

MVP-8: Advanced Statistical Analysis Workspace imekamilika.

Sasa StatFlow ina workflow ya karibu na statistical software halisi:

```
Data
↓
Clean
↓
Transform
↓
Analyze
↓
Validate
↓
Run Analysis
↓
Statistical Results
↓
Interpretation-ready Output
```

Vipengele vipya

- **Analysis Workspace** ndani ya web app
- Method recommendation kulingana na variable types
- Variable validation
- Complete-case/sample-size validation
- Missingness diagnostics
- Independent Welch t-test
 - p-value
 - mean difference
 - 95% CI
 - Cohen's d
 - Levene variance diagnostic
- Correlation
 - Pearson/Spearman
 - p-value
 - significance interpretation
- Descriptive statistics
- Frequency analysis
- Chi-square
- One-way ANOVA
 - F-statistic
 - p-value
 - η^2
 - group means + CI
- Linear regression
 - R^2
 - adjusted R^2
 - coefficients
 - SE
 - p-values
 - CI
 - AIC/BIC
 - Durbin-Watson

Pia result schema sasa inaanza kuwa standardized ili baadaye **AI Assistant** iweze kusoma results kutoka statistical engine badala ya kufanya calculations yenyewe.

Download

[Download StatFlow MVP-8](#)

Hatua inayofuata

MVP-9: AI Statistical Assistant ndiyo hatua kubwa inayofuata:

User: *“Je, kuna tofauti ya kipato kati ya wanaume na wanawake?”*

StatFlow itaweza:

```
User Question
    ↓
Understand Question
    ↓
Identify Variables
    ↓
Detect Variable Types
    ↓
Recommend Statistical Method
    ↓
Check Assumptions
    ↓
Run Real Statistical Engine
    ↓
Get Verified Results
    ↓
AI Explains Results
```

Muhimu: AI haitafanya calculation yenyewe; **statistical engine** ndiyo itafanya **calculation**, AI itaelezea matokeo yaliyothibitishwa.

Endelea

Worked for 47s

MVP-9: AI Statistical Assistant imekamilika.

Sasa StatFlow ina layer ya AI inayokaa juu ya statistical engine:

```
User's Question
    ↓
AI Question Understanding
    ↓
Identify Variables
    ↓
Detect Variable Types
    ↓
Select / Recommend Method
    ↓
Statistical Validation
```

↓
Real Statistical Engine
↓
Verified Results
↓
AI Explanation

Kilichoongezwa

- **AI Assistant UI**
- `/api/v1/ai/ask`
- `/api/v1/ai/suggest-analysis`
- `/api/v1/ai/explain-result`
- Automatic identification ya variables kutoka kwenye swali
- Recommendation ya:
 - Descriptive statistics
 - Frequency
 - Correlation
 - Group comparison
- AI explanation ya p-value na results
- AI haitumii LLM kufanya calculations; calculations zinafanywa na statistical engine.
- AI response ina:
 - Analysis plan
 - Variables zilizotambuliwa
 - Method
 - Statistical result
 - Interpretation

Mfano user anaweza kuuliza:

“Is there a relationship between income and farm size?”

System inaweza kutambua:

```
Outcome: income
Predictor: farm_size
Method: correlation
↓
Pearson correlation
↓
r, p-value
↓
AI interpretation
```

Nimepia kufanya **Python compile validation**, na API files zimepita syntax check.

Download MVP-9

Download StatFlow MVP-9 – AI Statistical Assistant

Hatua inayofuata: MVP-10

Sasa tunaweza kuanza sehemu muhimu sana ya product:

Research Mode + Automated Statistical Reporting

Itakuwa:

```
Research Question
    ↓
Objectives
    ↓
Hypotheses
    ↓
Variables
    ↓
Recommended Methods
    ↓
Assumption Tests
    ↓
Statistical Results
    ↓
Tables
    ↓
Charts
    ↓
Interpretation
    ↓
Research Report
```

Hii itaifanya StatFlow iwe na uwezo wa kusaidia **students, researchers, statisticians, NGOs, businesses na institutions** kutoka kwenye raw dataset mpaka kwenye report yenye statistical results.

Endelea

Worked for 36s

MVP-10: Research Mode + Automated Reporting imekamilika.

Sasa StatFlow inaanza kuwa zaidi ya statistical calculator; inaweza kufuata workflow ya research:

```
Research Study
  ↓
Objectives
  ↓
Hypotheses
  ↓
Variables
  ↓
Statistical Analysis
  ↓
Results
  ↓
Tables / Charts
  ↓
Research Report
```

Vipengele vilivyoongezwa

- **Research Project**
- Research title na description
- **Objectives management**
- **Hypotheses management**
 - H_0
 - H_1
- Objective → hypothesis linkage foundation
- Objective → variables → statistical method linkage
- Ku-attach statistical results kwenye objective
- **Automated HTML research report**
- Report inakusanya:
 - Title
 - Description
 - Objectives
 - Hypotheses
 - Analysis results
 - Statistical metrics
 - Timestamp
- Research Mode UI ndani ya web application

API mpya:

```
POST /api/v1/research
GET  /api/v1/research/{id}

POST /api/v1/research/{id}/objectives
POST /api/v1/research/{id}/hypotheses
```

```
POST /api/v1/research/{id}/links
POST /api/v1/research/{id}/analyses

GET /api/v1/research/{id}/report
```

Download

Download StatFlow MVP-10 — Research & Reporting

Nimefanya pia Python compile check ya backend; imepita bila syntax errors.

Hatua inayofuata: MVP-11

Sasa ningependekeza tujenge **Production Data Architecture + Big Data Engine**:

```
CSV/XLSX
  ↓
Parquet
  ↓
Polars LazyFrame / DuckDB
  ↓
Large Dataset Processing
  ↓
Server-side Query
  ↓
Paginated UI
```

Hapo tutaacha kutegemea kusoma dataset yote kwenye RAM kwa kila operation, jambo ambalo ni muhimu sana kwa vision yako ya **fast big-data analysis**.

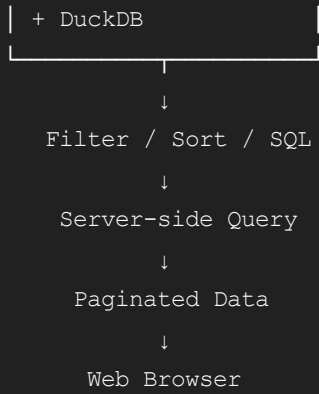
Endelea

MVP-11: Big Data Engine imekamilika.

Sasa architecture ya StatFlow inaanza kuelekea kwenye uwezo wa kuchambua datasets kubwa bila kujaribu kupakia kila row kwenye browser au LLM.

Architecture mpya

```
CSV / XLSX
  ↓
Parquet
  ↓
┌───────────────────┐
│ Polars Lazy Engine │
```



Kilichoongezwa

1. Polars LazyFrame

Badala ya:

Python

Run

```
df = read_entire_dataset()
```

tunatumia:

Python

Run

```
pl.scan_parquet(...)
```

Hii inaruhusu operations kucheleweshwa na kusoma columns/rows zinazohitajika tu.

2. DuckDB

Imeongezwa kwa analytical SQL dhidi ya Parquet:

SQL

```
SELECT gender, AVG(income)
FROM {dataset}
GROUP BY gender
```

3. Server-side pagination

Browser haihitaji kupokea dataset nzima.

```
Page 1 → rows 1-100
Page 2 → rows 101-200
Page 3 → rows 201-300
...
```

4. Server-side sorting

Sorting kubwa inafanyika backend badala ya browser.

5. Dataset profiling

Engine inaweza kupata:

- row count
- column count
- column names
- data types

6. Security boundary

SQL endpoint inaruhusu `SELECT/WITH` pekee, si arbitrary write operations.

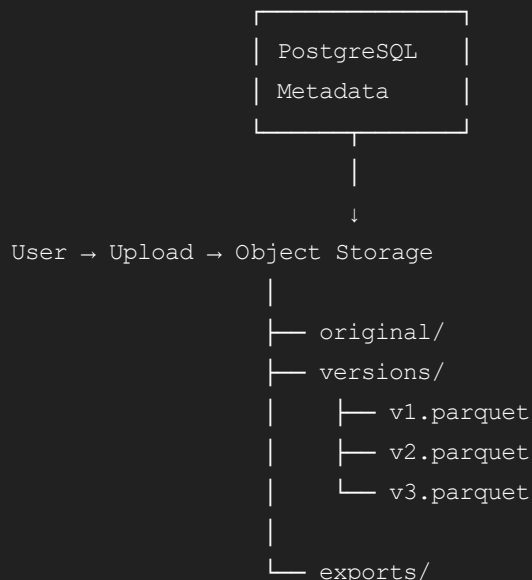
Download

Download StatFlow MVP-11 — Big Data Engine

Hatua inayofuata

MVP-12: Production Data Architecture ndiyo muhimu sasa.

Tutabadilisha storage kuwa:

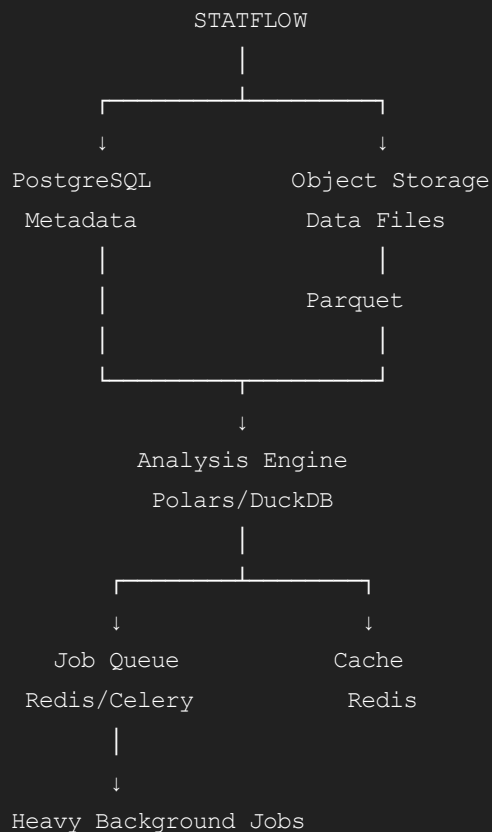


na kuweka **background jobs**, **caching**, **dataset versioning**, **concurrent processing** na **large-file upload handling**.

Hapo ndipo tutaanza kuwa na msingi unaoweza kubeba datasets kubwa zaidi kwa production. ☐

MVP-12: Production Data Architecture imekamilika.

Sasa StatFlow ina architecture inayofaa zaidi kwa production:



Vipengele vipya

- Dataset lifecycle:
 - UPLOADING
 - PROCESSING
 - READY
 - FAILED
- Immutable dataset versions
- Object-storage abstraction tayari kwa S3-compatible storage
- Original file separation
- Parquet version storage
- Background-job abstraction tayari kwa Redis/Celery
- Analysis caching
- Dataset-version-based cache keys

- PostgreSQL metadata boundary
- Storage layout ya production

Mfano:

```
datasets/  
└─ organization/  
    └─ project/  
        └─ dataset/  
            ├── original/  
            │   └─ survey.xlsx  
            ├── versions/  
            │   ├── v1.parquet  
            │   ├── v2.parquet  
            │   └─ v3.parquet  
            └─ exports/
```

Download

[Download StatFlow MVP-12 — Production Architecture](#)

Hatua inayofuata: MVP-13

Sasa ningesogeza mfumo kwenye **Authentication + Organizations + Projects + Permissions:**

```
User  
↓  
Organization  
↓  
Project  
↓  
Dataset  
↓  
Analysis  
↓  
Reports
```

ikiwa na:

- Register/Login
- JWT access + refresh tokens
- User profile
- Organizations
- Organization members
- Roles
- Projects
- Dataset ownership

- Permissions
- Audit trail

Hii ni muhimu kabla hatujaanza kuweka StatFlow online, kwa sababu kila dataset, analysis na report lazima ijulikane **ni ya nani na nani ana ruhusa ya kuiona au kuibadilisha.** ☐

Endelea

It seems like I can't do more advanced data analysis right now. Please try again later. ☐

ChatGPT can make mistakes. Check important info.